

Claude Certified Architect — Foundations Certification

مطالعہ گائیڈ (سرکاری امتحانی گائیڈ کی بنیاد پر)

تعارف

سرٹیفیکیشن اس بات کی تصدیق کرتی ہے کہ ایک ماہر حقیقی Claude Certified Architect — Foundations Claude Code، Claude Agent SDK، Claude API، اور Model Context Protocol (MCP) — یعنی بنیادی علم کا جائزہ لیتا ہے — کے ساتھ پروڈکشن ایپلیکیشنز بنائی جاتی ہیں۔ Claude و بنیادی ٹیکنالوجیز جن سے

agentic systems امتحانی سوالات حقیقت پسندانہ صنعتی منظرناموں پر مبنی ہوتے ہیں: کسٹمر سپورٹ کے لیے، multi-agent research pipelines، ڈیزائن کرنا، Claude Code کو CI/CD میں ضم کرنا، developer productivity، structured data بنانا، اور غیر ساختہ دستاویزات سے tools نکالنا۔

هدف امیدوار

کرتا ہے آپ ship کے ساتھ پروڈکشن ایپلیکیشنز ڈیزائن اور Claude کے جو solution architect مثالی امیدوار ایک کے پاس کم از کم 6 ماہ کا عملی تجربہ ہونا چاہیے:

- Claude Agent SDK** — multi-agent orchestration، subagents کو delegate کرنا، tool integration، lifecycle hooks
- Claude Code** — CLAUDE.md، MCP servers، Agent Skills، planning mode
- Model Context Protocol (MCP)** — backend integration کے لیے tools اور resources
- Prompt engineering** — JSON schemas، few-shot examples، data extraction templates
- Context windows** — multi-agent context passing، لمبی دستاویزات کے ساتھ کام
- CI/CD pipelines** — automated code review، test generation
- Escalation and reliability** — error handling، human-in-the-loop

امتحانی فارمیٹ

پیرامیٹر	قدر
سوال کی قسم	Multiple choice (میں سے 1 درست 4)
اسکورنگ	100–1000 scale، passing score 720
Guessing penalty	نہیں (سوال کا جواب دیں!)
Scenarios	8 4 (randomly selected) میں سے

امتحانى مواد: 5 ڈومينز

وزن	ڈومین
27%	1. Agent architecture and orchestration
18%	2. Tool design and MCP integration
20%	3. Claude Code configuration and workflows
20%	4. Prompt engineering and structured output
15%	5. Context management and reliability

امتحان منظر نامہ ☐

Scenario 1: Customer Support Agent

account اور، returns، billing disputes استعمال کرتے ہوئے Claude Agent SDK بناتے ہیں جو آپ ایک agent MCP tools (`get_customer`، `lookup_order`، `process_refund`، `escalate_to_human`) سنبھالتا ہے۔ مناسب first-contact resolution +80% استعمال کرتا ہے۔ 80 دفعہ سے زیادہ۔

Scenario 2: Claude Code Code Generation

code generation, refactoring, debugging, documentation اور custom slash commands کے ساتھ، آپ کو اسے development میں استعمال کرتے ہوئے Claude Code کے ساتھ integrate کرنا اور planning mode کرنا، اور سمجھنا کے integrate

Scenario 3: Multi-Agent Research System

تیار کرنا، جاننا، reports کو مکمل citations سسٹم کو report generation اور synthesis tasks کو specialized subagents اور coordinator agent سونپنا

Scenario 4: Developer Productivity Tools

□ agent engineers کو unfamiliar codebases explore □، boilerplate code اور routine tasks بنانا،
استعمال MCP servers اور Built-in tools (Read, Write, Bash, Grep, Glob) میں مدد دیتا □□□
□ کد □ جاز □ □

Scenario 5: Claude Code for Continuous Integration

pull اور automated code reviews، test generation میں ضم کریں تاکہ CI/CD pipeline کو Claude Code request feedback کم سے کم false positives ڈیزائن ہونے کا نثر کے Prompts حاصل ہوں۔

Scenario 6: Structured Data Extraction

کرتا ہے، validate کے ذریعے JSON schemas کو output، سسٹم غیر ساختہ دستاویزات سے معلومات نکالتا ہے اور high accuracy کے لیے edge cases پر سنبھالنا چاہیے۔ اس کا برقرار رکھنا اس کے لیے ضروری ہے۔

Scenario 7: Conversational AI Architecture Patterns

multi-turn conversational systems میں جن میں context window management، turns متعدد، memory strategies، tool design کے لیے execution محفوظ، کا برقرار رکھنا instructions میں اور مہم یا متضاد، user inputs کو سنبھالنا شامل ہے۔

Scenario 8: Agentic AI Tools (ہماری مدد کریں — مواد موجود نہیں)

میں شامل نہیں ہے۔ guide کے لیے ابھی تک اس candidates کی اطلاع امتحان دینے والے scenario اس میں share میں GitHub Issues کے سوالات دیکھیں، تو ان میں scenario کے اصل امتحان میں اس شامل کر سکیں۔ آپ کا تعاون اس شخص کی مدد کرے گا جو امتحان کی تیاری کے لیے coverage تاکہ مہم مکمل کرے۔

سرکاری دستاویزات

وسیلہ	URL
Claude API — Messages	https://platform.claude.com/docs/en/api/messages
Claude API — Tool Use	https://platform.claude.com/docs/en/build-with-claude/tool-use
Claude API — Message Batches	https://platform.claude.com/docs/en/build-with-claude/message-batches
Claude Agent SDK — Overview	https://platform.claude.com/docs/en/agent-sdk/overview
Claude Agent SDK — Hooks	https://platform.claude.com/docs/en/agent-sdk/hooks
Claude Agent SDK — Subagents	https://platform.claude.com/docs/en/agent-sdk/subagents
Claude Agent SDK — Sessions	https://platform.claude.com/docs/en/agent-sdk/sessions
Model Context Protocol (MCP)	https://modelcontextprotocol.io/
MCP — Tools	https://modelcontextprotocol.io/docs/concepts/tools
MCP — Resources	https://modelcontextprotocol.io/docs/concepts/resources
MCP — Servers	https://modelcontextprotocol.io/docs/concepts/servers
Claude Code — Documentation	https://code.claude.com/docs/en/overview
Claude Code — CLAUDE.md and Memory	https://code.claude.com/docs/en/memory
Claude Code — Skills (incl. slash commands)	https://code.claude.com/docs/en/skills

Claude Code — Hooks	https://code.claude.com/docs/en/hooks
Claude Code — Sub-agents	https://code.claude.com/docs/en/sub-agents
Claude Code — MCP Integration	https://code.claude.com/docs/en/mcp
Claude Code — GitHub Actions CI/CD	https://code.claude.com/docs/en/github-actions
Claude Code — GitLab CI/CD	https://code.claude.com/docs/en/gitlab-ci-cd
Claude Code — Headless (non-interactive mode)	https://code.claude.com/docs/en/headless
Prompt Engineering Guide	https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview
Extended Thinking	https://platform.claude.com/docs/en/build-with-claude/extended-thinking
Anthropic Cookbook (code examples)	https://github.com/anthropics/anthropic-cookbook

نظریاتی بنیادیں: حصہ 1

یہ حصہ تمام نظریاتی احاطہ کرتا ہے جس کی آپ کو امتحان میں کامیابی کے لیے ضرورت ہے۔ مواد کو کے مطابق — اس سے آپ کے domains کے مطابق ترتیب دیا گیا ہے، نہ کہ امتحانی concepts ٹیکنالوجیز اور موضوع کی زیادہ گہری سمجھ پیدا کر سکتے ہیں۔

کی بنیادیں Claude API — Model Interaction: باب 1

دستاویزات: [Messages API](#) | [Prompt Engineering](#)

1.1 API Request Structure

Claude API کی request کی ر Claude Messages API کی پیروی کرتا ہے request-response model ایک شامل ہوتا ہے:

```
{
  "model": "claude-sonnet-4-6",
  "max_tokens": 1024,
  "system": "You are a helpful assistant.",
  "messages": [
    {"role": "user", "content": "Hi!"},
    {"role": "assistant", "content": "Hello!"},
    {"role": "user", "content": "How are you?"}
  ],
  "tools": [...],
  "tool_choice": {"type": "auto"}
}
```

ا:م فیلڈز

- `model` — model selection (`claude-opus-4-6` , `claude-sonnet-4-6` , `claude-haiku-4-5`)
- `max_tokens` — response کی تعداد tokens میں زیادہ سے زیادہ
- `system` — system prompt (model کی تعریف کرتا ہے)
- `messages` — conversation history (تاریخچه گفتگو) (آپ کو مکمل history تسلسل برقرار رکھنا کہ لے آپ کو مکمل) (ہیجینی وئی history تسلسل برقرار رکھنا کہ لے آپ کو مکمل)
- `tools` — definitions کی tools دستیاب
- `tool choice` — tool selection strategy

1.2 Message Roles

استعمال کرتا □□ roles تین array messages

- `user` — user messages
- `assistant` — model responses (history میں شامل کیے جاتے ہیں)
- `tool` — tool call results (role پر واضح طور پر `set` کیا جاتا ہے؛ `tool_result` content block کی صورت میں دیا جاتا ہے)

Model requests بھیجی جائے گی۔ conversation history میں آپ کو مکمل API request اتنا ہی ملے گا:۔

1.3 Response میں stop_reason Field

Claude API response میں، `stop_reason` model شامل ہوتا ہے، جو بتاتا ہے کہ

Value	Description	Action
"end_turn"	Model کا response مکمل کر لیا	user کو دکھائیں نتیجہ
"tool_use"	Model tool call کرنا چاہتا ہے	Tool execute result دیں اور کریں
"max_tokens"	Token limit ختم ہو گئی	Response truncated ہو گیا; limit بڑھانی پڑے گی
"stop_sequence"	Stop sequence مل گئی	اپنی application logic کے مطابق handle کریں

کو کنٹرول کرتے agent loop سب سے اہم ہیں — یہی "end_turn" اور "tool_use" میں Agentic systems ہیں۔

1.4 System Prompt

System prompt متعین کرتا ہے behavioral rules اور context جو instruction ایک خاص

- messages array الگ ہوتا؛ یہ system field کا حصہ نہیں ہوتا؛ بلکہ الگ array
- user messages پر فوقیت رکھتا ہے
- میں لاگو رہتا ہے conversation ہوتا ہے اور پوری load ایک بار
- کی تعریف کے لیے استعمال ہوتا ہے output format اور role, constraints,

پیدا کر سکتی ہے مثال کے طور پر tool associations غیر ارادی wording کی system prompt: امتحان کے لیے اہم کو ضرورت سے زیادہ get_customer کو model کی تصدیق کریں "جیسی ہدایت customer پر،" ہمیشہ استعمال کرنے پر مجبور کر سکتی ہے، حتیٰ کے جب یہ ضروری نہ ہو

1.5 Context Window

Context window text (tokens) جسے اس میں process ایک وقت میں model کی کل مقدار ہے جسے شامل ہے:

- System prompt
- message history مکمل
- Tool definitions
- Tool results

مسائل context-window:

1. **Lost-in-the-middle effect:** models input سے اعتماد سے زیادہ موجود معلومات پر اختتام اور آغاز کے process کر سکتے ہیں۔ حل: اہم معلومات شروع یا آخر کے قریب miss کرتے ہیں، مگر درمیان کی تفصیلات رکھیں
2. **Tool results** واپس کر کے tool 40+ fields شامل کرتا ہے اگر output میں tool call context کا جمع ہونا: ہر Tool results کا بڑا حصہ ضائع ہو جاتا ہے context مگر صرف 5 اہم ہوں، تو
3. **Progressive summarization:** history compress کرتے وقت numeric values, percentages, اور dates اکثر گم ہو کر غیر واضح ہو جاتے ہیں ("تقریباً"، "کچھ"، "چند")

2 Tools اور tool_use: باب

Tool Use: دستاویزات

2.1 tool_use کیا

برا Model کی اجازت دیتا Claude کو external functions call جو mechanism ایک tool_use result کرتا اور execute اس code بناتا؛ آپ کا structured tool call request میں چلاتا — و code راست واپس کرتا

2.2 Tool Definition

کیا جاتا define کے ذریعہ JSON schema ایک tool

```
{
  "name": "get_customer",
  "description": "Finds a customer by email or ID. Returns the customer profile, including name, email, order history, and account status. Use this tool BEFORE lookup_order to verify the customer's identity. Accepts an email (format: user@domain.com) or a numeric customer_id.",
  "input_schema": {
    "type": "object",
    "properties": {
      "email": {"type": "string", "description": "Customer email"},
      "customer_id": {"type": "integer", "description": "Numeric customer ID"}
    },
    "required": []
  }
}
```

کے انتہائی اہم ہلو Tool description:

- Description tool selection** کی بنیاد پر descriptions کو ان کی LLM tools mechanism کا بنیادی Minimal descriptions ("Customer information retrieve کرتا) اُن مواقع پر غلطیوں کا (کرتا) overlap کے ایک دوسرے tools باعث بنتی ہیں جب کرتے ہوں
- Description میں شامل کریں:**
 - کیا کرتا اور کیا واپس کرتا Tool
 - Input formats اور example values
 - Edge cases اور constraints
 - مقابلہ میں کب استعمال و tool similar alternatives
- کی analyze_document اور analyze_content سے بچیں اگر overlapping descriptions ایک جیسے یا انہیں خلط ملط کر دے گا model تقریباً ایک جیسی ہوں، تو descriptions
- Built-in tools** MCP tools: agents similar functionality والے built-in tools (Read, Grep) کو MCP tools — مضبوط بنائیں MCP tool descriptions پر ترجیح دے سکتے ہیں اس سے بچنے کے لیے concrete advantages, unique data, context یا و built-in tools میں کر سکتے ہیں جو

2.3 tool_choice Parameter

کیسے منتخب کرتا model tools کے کنٹرول کرتا tool_choice

Value	Behavior	کب استعمال کریں
<code>{ "type": "auto" }</code>	Model کرنا tool call خود ط کرتا یا کرنا text یا میں جواب دینا	default زیاد تر حالات میں
<code>{ "type": "any" }</code>	Model کرنا tool call کو لازماً کوئی	guaranteed جب آپ کو structured output چاہیں
<code>{ "type": "tool", "name": "extract_metadata" }</code>	Model کرنا tool call کو لازماً ایک مخصوص tool کو	forced first step / execution order جب آپ کو چاہیں

ا: م منظر نامہ:

- `tool_choice: "any"` + multiple extraction tools → model منتخب کرتا ہے، مگر پھر بھی tool بہترین ترین ملتا structured output
- Forced selection → enrichment (مثلاً) کی ضمانت چاہیں action جب آپ کو کسی خاص پم (`extract_metadata`)

2.4 Structured Output JSON Schemas

حاصل کرنے کا سب سے قابلِ structured output Claude استعمال کرنا `tool_use` کے ساتھ JSON schemas

اعتماد طریقہ کے:

- Syntax (trailing commas، غائب نہیں ہوتے braces) کی ضمانت دیتا ہے JSON کے اعتبار سے درست (ہوتے)
- (موجود ہوتے ہیں required fields) نافذ کرتا ہے structure مطلوبہ
- Semantic correctness (values) کی ضمانت نہیں دیتا

Schema design — بنیادی اصول:

```

{
  "type": "object",
  "properties": {
    "category": {
      "type": "string",
      "enum": ["bug", "feature", "docs", "unclear", "other"]
    },
    "category_detail": {
      "type": ["string", "null"],
      "description": "Details if category = 'other' or 'unclear'"
    },
    "severity": {
      "type": "string",
      "enum": ["critical", "high", "medium", "low"]
    },
    "confidence": {
      "type": "number",
      "minimum": 0,
      "maximum": 1
    },
    "optional_field": {
      "type": ["string", "null"],
      "description": "Null if the information was not found in the source"
    }
  },
  "required": ["category", "severity"]
}

```

Schema design rules:

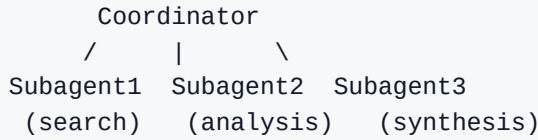
- Required vs optional:** بنائیں جن کی معلومات ہمیشہ دستیاب ہو۔ Required fields کو fields صرف اُن optional: Required fields model کو data پر values غائب ہونے پر مجبور کر سکتی ہیں۔
- Nullable fields:** استعمال کریں جو ممکنہ طور پر `"type": ["string", "null"]` ایسی معلومات کے لیے۔ بنائے value واپس کر سکتا ہے بجائے اس کے کہ وہ خود ساختہ `null` Model موجود نہ ہوں۔
- Enums with "other":** predefined categories کے لیے ہجاء کے لیے `"other"` شامل کریں۔ `"other"` + detail string
- Enum "unclear":** `"unclear"` کے بارے میں پُر اعتماد نہ ہو — ایماندار model category اُن صورتوں کے لیے جب `"unclear"` سے بہتر category غلط

2.5 Syntax vs Semantic Errors

Error type	Example	Mitigation
Syntax	Invalid JSON, field type غلط	JSON schema کے ساتھ <code>tool_use</code> (ختم کر دیتا) <code>tool_use</code>
Semantic	Totals match نہیں کرتے، value field غلط، hallucination	Validation checks, feedback کے ساتھ <code>retry</code> , self-correction

3.3 Hub-and-Spoke: Coordinator اور Subagents

Multi-agent architecture عموماً hub-and-spoke topology میں بنائی جاتی ہے:



Coordinator کی ذمہ داریاں:

- Task کو subtasks میں تقسیم کرنا
- (dynamic selection) درکار subagents میں سے کون سا طے کرنا
- کو سونپنا subagents کام
- کرنا validate نتائج جمع اور
- Errors اور retries handle کرنا
- تک پہنچانا user نتائج

الگ ہوتا context کا subagents: اصول

- Subagents conversation history کی coordinator خود بخود inherit نہیں کرتے
- میں واضح طور پر دینا ہوتا context subagent prompt تمام ضروری
- Subagents memory share کے درمیان calls نہیں کرتے
- (observability اور error control کے ذریعے گزرتی ہے coordinator communication تمام)

3.4 Subagents کے لیے Task Tool بنانا

Subagents Task tool کے ذریعے spawn میں لے جاتے ہیں:

```
# Coordinator میں allowedTools کے "Task" ہونا چاہیے
coordinator_agent = AgentDefinition(
    allowed_tools=["Task", "get_customer"]
)
```

Explicit context passing لازمی ہے:

```
# Bad: subagent کے پاس context نہیں ہے
Task: "Analyze the document"

# Good: context prompt مکمل میں
Task: "Analyze the following document.
Document: [full document text]
Prior search results: [web search results]
Output format requirements: [schema]"
```

Parallel spawning: coordinator ایک response میں متعدد Task s call کر سکتا ہے — subagents parallel چلتے ہیں:

```
# Coordinator کے ایک response میں شامل ہے
Task 1: "Search for articles about X"
Task 2: "Analyze document Y"
Task 3: "Search for articles about Z"
# تینوں ایک ساتھ چلتے ہیں
```

3.5 Agent SDK میں Hooks

Hooks agent lifecycle میں transformation اور interception پر points کے مخصوص

PostToolUse tool result کو model سے intercept تک پہنچانے سے پہلے model کو PostToolUse

```
# MCP tools کے تاریخ سے formats normalize کریں
@hook("PostToolUse")
def normalize_dates(tool_result):
    # Unix timestamp -> ISO 8601
    # "Mar 5, 2025" -> "2025-03-05"
    return normalized_result
```

Outgoing-call interception hook policy والے actions کی خلاف ورزی کرنے والے کو block

```
# refunds block مثال: $500 سے اوپر
@hook("PreToolUse")
def enforce_refund_limit(tool_call):
    if tool_call.name == "process_refund" and tool_call.args.amount > 500:
        return redirect_to_escalation(tool_call)
```

Hooks اور prompt instructions میں فرق

Attribute	Hooks	Prompt instructions
Guarantee	Deterministic (100%)	Probabilistic (>90%, 100% نہیں)
کب استعمال کریں	Critical business rules, financial operations, compliance	General preferences, recommendations, formatting
مثال	Refunds > \$500 block کریں	"Try to solve before escalating"

استعمال کریں hooks، انہیں prompts — کے مالی، قانونی، یا حفاظتی نتائج کے failure قاعدے: جب

4: Model Context Protocol (MCP)

دستاویزات: [MCP](#) | [Tools](#) | [Resources](#) | [Servers](#)

4.1 MCP کیا ہے

MCP (Model Context Protocol) Claude کو external systems سے جوڑتا ہے جو open protocol ایک ہے۔ MCP سروس جوڑتا ہے جو external systems سے جوڑتا ہے جو open protocol ایک ہے۔ MCP سروس جوڑتا ہے جو external systems سے جوڑتا ہے جو open protocol ایک ہے۔

1. **Tools** — functions (agent actions) کے لیے (CRUD operations, API calls, command execution)
2. **Resources** — context کے لیے (documentation, database schemas, content catalogs)
3. **Prompts** — predefined prompt templates کے لیے tasks عام

4.2 MCP Servers

MCP server ایک process جو MCP protocol implement کرتا ہے اور tools/resources جب فراہم کرنا چاہے۔ MCP server آپ سے connect کرے گا:

- وہ خود بخود tools discover کرے گا
- ایک ساتھ دستیاب ہونے والے tools کے connected servers تمام
- انہیں کیسے استعمال کرنا ہے گا model طے کرتی ہے

4.3 MCP Servers کو Configure کرنا

Project configuration (`.mcp.json`) — team usage کے لیے:

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_TOKEN": "${GITHUB_TOKEN}"
      }
    },
    "jira": {
      "command": "npx",
      "args": ["-y", "mcp-server-jira"],
      "env": {
        "JIRA_TOKEN": "${JIRA_TOKEN}"
      }
    }
  }
}
```

ملاحظات:

- میں ہوتا ہے version control میں محفوظ ہوتا ہے اور `.mcp.json` project root
- Environment variables (`${GITHUB_TOKEN}`) secrets میں استعمال ہوتے ہیں — tokens خود commit نہیں کیے جاتے

- کھ لیکھ دستياب ٿوٽا contributors کھ تمام project ٿي

User configuration (~/ .claude.json) — personal/experimental servers کھ لیکھ:

- User کھ home directory کھ ٿوٽا کھ محفوظ
- Version control کھ ذريعہ کھ share ٿوٽا کھ
- Personal experiments کھ لیکھ موزون testing اور

Servers کھ انتخاب:

- Standard integrations (Jira, GitHub, Slack) کھ موجود کھ لیکھ پڻ کھ سدھ کھ موجود کھ لیکھ
- کھ لیکھ بنائين unique, team-specific workflows کھ صرف custom servers کھ اپني

4.4 MCP کھ ميں isError Flag

کھ بتاتا کھ کھ agent کھ استعمال ٿوٽا کھ لیکھ isError: true کھ ميں response آتا کھ ٿو error ميں MCP tool کھ جب کھ ٿوئي کھ call fail کھ

Structured error (اچھا):

```
{
  "isError": true,
  "content": {
    "errorCategory": "transient",
    "isRetryable": true,
    "message": "The service is temporarily unavailable. Timeout while calling the orders API.",
    "attempted_query": "order_id=12345",
    "partial_results": null
  }
}
```

Generic error (anti-pattern):

```
{
  "isError": true,
  "content": "Operation failed"
}
```

Generic error agent کھ کيا کھ escalade بدليں، يا query، کھريں retry کھ فيصلہ کھ سازي کھ لیکھ کھ ٿوئي معلومات کھ ٿيڻ ديتا — کھ کيا کھ Generic error کھ کھريں؟

4.5 MCP Resources

Resources کھ کھيڻ بغير طلب کھ سکندا کھ کھ actions کھ حاصل کھريڻ کھ لیکھ agent context کھ ٿيڻ جو ايسا data کھ

- Content catalogs (hierarchical navigation، کھي فہرست کھ project tasks مثلاً تمام)

- Database schemas (data structure سمجھنے کے لیے)
- Documentation (API references, internal guides)
- Issue/task summaries

Resource کا فائدہ: agent کے لیے exploratory tool calls میں پڑتی کے سمجھنے کے لیے agent کا فائدہ: Resource کا فائدہ: map فوری Resource کیا ہے

5 اور Claude Code — Configuration Workflows

دستاویزات: [Claude Code](#) | [Memory / CLAUDE.md](#) | [Skills](#) | [MCP](#) | [Hooks](#) | [Sub-agents](#) | [GitHub Actions](#) | [Headless](#)

5.1 CLAUDE.md Hierarchy

hierarchy استعمال کرتا ہے اس کی تین سطحی Claude Code میں جو instruction file(s) و CLAUDE.md ہے:

1. User-level: ~/.claude/CLAUDE.md
 - پر لاگو user صرف اسی
 - نمونہ ہوتا ہے share کہ ذریعہ VCS
 - working style اور preferences ذاتی
2. Project-level: .claude/CLAUDE.md یا root CLAUDE.md
 - پر لاگو project contributors تمام
 - ہوتا ہے manage کہ ذریعہ VCS
 - Coding standards, testing standards, architectural decisions
3. Directory-level: subdirectories میں CLAUDE.md
 - کہ ساتھ کام ہو تو لاگو ہوتا ہے files کی directory جب اسی
 - conventions کہ اُس حصہ کہ مخصوص codebase

~/.claude/CLAUDE.md میں ملتی کیونکہ و project instructions کو team member عام غلطی: ایک نئے میں رکھی گئی تھیں (user-level) ~/.claude/CLAUDE.md (project-level) کے بجائے

5.2 @path Syntax (File Imports)

جو جاتی configuration modular کر سکتا ہے، جس سے refer کے ذریعہ @path کو files بیرونی CLAUDE.md ہے:

Coding standards @./standards/coding-style.md میں بیان کیے گئے ہیں
 Test requirements @./standards/testing-requirements.md میں ہیں
 Project overview @README.md اور dependencies @package.json میں ہیں

- (نہ) میں space کوئی) آئے file path کے فوراً بعد @
- Relative اور absolute دونوں paths supported ہیں
- Relative paths کے مطابق file اُس import کے وٹے ہیں جس میں resolve کے مطابق file اُس
- Maximum import nesting depth 5 ہے

شامل ہوتی ہیں، standards میں صرف متعلقہ package کم ہوتی ہیں اور duplication اس سے

کرنے کے organize کے حساب سے topic کو rules کا متبادل ہے، جو monolithic CLAUDE.md .claude/rules/

لیہ استعمال ہوتا ہے:

```
.claude/rules/  
  testing.md          -- testing conventions  
  api-conventions.md  -- API conventions  
  deployment.md       -- deployment rules  
  react-patterns.md   -- React patterns
```

```
---
paths: ["src/api/**/*.json"]
---
```

استعمال کریں۔ explicit error handling کہ ساتھ async/await کہ لیے API files
وایس کہ endpoint standard response wrapper

```
---
paths: ["**/*.test.tsx", "**/*.test.ts"]
---
```

استعمال ہونہ چاہئیں۔ describe/it blocks میں Tests
 ہمیں، hardcoding، استعمال کریں Data factories
 استعمال کریں، test database - نہ کریں Database mock

۱۰. کس کا کام کرتا ہے؟

- کرتا match سہ pattern paths کر جو file edit ایسا Claude Code ہوتا ہے جب load صرف اس وقت Rule ہو
- نہیں ہیں rules load ہوتے ہیں — غیر متعلقہ tokens اور context اس سہ
- Glob patterns کرنے دینے ہیں، چاہے conventions apply کے لحاظ سہ file type آپ کو میں codebase (کے لیے بہت مفید tests) کے ہیں بھی ہوں

کب استعمال کریں directory-level CLAUDE.md بمقابلہ .claude/rules/ والے paths

- .claude/rules/ with paths — جب conventions کئی directories میں بکھری files (tests, migrations) پر لاگو ہوں
- Directory-level CLAUDE.md — جب conventions کسی خاص directory میں اور کے ہیں اور ضروری نہ ہوں

5.4 Custom Slash Commands اور Skills

skills کو (.claude/commands/) custom commands میں Claude Code version نوٹ: موجود بناؤ commands /name formats کر دیا گیا ہے دونوں unify کے ساتھ (.claude/skills/) supported اب بھی format کا ذکر کرتی ہے — و (.claude/commands/) میں امتحانی گائیڈ

Slash commands reusable prompt templates ہیں جو /name کے ذریعے invoke کے ہیں:

.claude/commands/ format (legacy, supported):

```
.claude/commands/
review.md      -- /review -- standard code review
test-gen.md    -- /test-gen -- test generation
```

.claude/skills/ format (current):

```
.claude/skills/
review/SKILL.md -- /review -- frontmatter configuration
test-gen/SKILL.md
```

Project commands (.claude/commands/ یا .claude/skills/):

- کرنے والے سب کے لیے دستیاب ہوتے ہیں repo clone میں محفوظ ہوتے ہیں اور VCS
- یقینی بناتے ہیں consistent workflows ٹیم میں

User commands (~/.claude/commands/ یا ~/.claude/skills/):

- نہیں ہیں ہوتے share کے ذریعے VCS جو commands ذاتی
- کے لیے workflows انفرادی

5.5 Skills — .claude/skills/

Skills advanced commands ہیں جو SKILL.md frontmatter کے ذریعے configure کے ہیں:

```
---
context: fork
allowed-tools: ["Read", "Grep", "Glob"]
argument-hint: "Path to the directory to analyze"
---
```

کا تجزیہ کریں۔ code structure میں directory دی گئی
report پر ایک architectural patterns اور Dependencies

Frontmatter parameters:

Parameter	Description
<code>context:</code> <code>fork</code>	Verbose output main session کو pollute کرتا isolated subagent کو Skill
<code>allowed-</code> <code>tools</code>	نہیں کر skill files delete مثلاً — security) دستیاب ہوں گے tools یہ محدود کرتا ہے کہ کون سے (سکے گی اگر اجازت نہ ہو)
<code>argument-</code> <code>hint</code>	مانگنے کا اشارہ argument کو تو invoke کے بغیر skill parameters جب

Skill `CLAUDE.md` کے استعمال کریں:

- Skill — task مخصوص — on-demand invocation (review, analysis, generation) کے لیے
- `CLAUDE.md` — general standards اور conventions ہمیشہ لوڈ ہونے والے

Personal skills (`~/claude/skills/`):

- متاثر نہ ہوں teammates مختلف ناموں سے بنائیں تاکہ personal variants اپنی

5.6 Planning Mode `Direct Execution` کے مقابلہ

Planning mode:

- Model نہایت changes کرتا ہے؛ plan اور investigate صرف
- Codebase explore کے لیے Read, Grep, Glob استعمال کرتا ہے
- کرتا ہے user approve تیار کرتا ہے جس implementation plan ایک
- safe exploration کے side effects بغیر

Planning mode کے استعمال کریں:

- changes (files درجنوں) بڑے
- Multiple plausible approaches (microservices: boundaries کیسے define ہوں؟)
- Architectural decisions (کیا framework کون سا؟ structure؟)
- Unfamiliar codebase (change کی ضرورت ہے؟ سمجھنا ضروری ہے؟)
- Library migrations 45+ files کو متاثر کریں

Direct execution کے استعمال کریں:

- Single-file fixes ساتھ stack trace واضح
- شامل کرنا validation check ایک
- changes اچھی طرح سمجھ گئی، غیر مبہم

Combined approach:

1. Investigation اور design کے لیے planning mode
2. User plan approve کر
3. Approved plan implement کے لیے direct execution کرنے

Explore subagent — codebase explore کے لیے specialized subagent:

- Verbose output کو main context سے isolate کرنا
- واپس کرنا summary صرف
- روکنا Multi-phase tasks میں context-window exhaustion

5.7 /compact Command

/compact context compress کے لیے built-in command:

- میں جگہ خالی کرنا context window کو summarize کر کے history چھپی
- Long investigation sessions جب verbose tool output سے context بھر جائے
- Risk: exact numeric values, dates, اور specific details summarization میں گم ہو سکتے ہیں

5.8 /memory Command

/memory sessions memory manage کے لیے built-in command:

- محفوظ کیے جا سکیں context اور notes, preferences, کے لیے کھولنا، تاکہ CLAUDE.md file editing
- Information sessions اور load پر خود بخود startup کے پار برقرار رہتی ہیں
- Project conventions, user preferences, frequently used commands, اور current work context محفوظ کر کے مفید
- دوبارہ سمجھانے کا متبادل instructions میں واپسی session

5.9 Claude Code CLI for CI/CD

-p (یا --print) flag:

```
claude -p "Analyze this pull request for security issues"
```

- Non-interactive mode: prompt process کرنا، اور print پر stdout، exit کرنا،
- User input کا انتظار نہیں کرتا
- CI/CD pipelines میں Claude کا واحد درست طریقہ

CI کے لیے structured output:

```
claude -p "Review this PR" --output-format json --json-schema
'{"type":"object",...}'
```

- `--output-format json` — output کو JSON میں دیتا ہے
- `--json-schema` — output کو schema مطابق validate کرتا ہے
- `Result` کو automatically inline PR comments میں لے کر پوسٹ کر دیا جاتا ہے

Session context isolation: میں کم مؤثر review کیا، و، اکثر code generate جس نہ Claude session (model) کے review کا امکان کم ہوتا ہے۔ چیلنج ساتھ رکھتا اور اپنے فیصلوں کو reasoning context (اپنی model) سے Review کے لیے independent instance استعمال کریں۔

review results context کرتا وقت، پچھل review کے بعد دوبار commits سہ بچاؤ: نئے Duplicate comments
 کرنے کی ہدایت دیں unresolved issues report کو صرف نئے یا Claude میں شامل کریں اور

5.10 fork_session اور Session Management

`--resume <session-name>` named session کو resume کرنا:

```
claude --resume investigation-auth-bug
```

- saved context conversation □□ جاری رکھتا □□ ساتھ، پچھلی
- Long investigations sessions مفید □□ لی□□ جو متعدد
- Risk: □□ □□ یں tool results stale بدل چکی □□ تو files □□ بعد session اگر پچھلی

fork_session shared context □□ independent branch □□ بنانا:

```
graph TD; A[Codebase investigation] --> B[fork_session]; B --> C[Approach A: Redux]; B --> D[Approach B: Context API];
```

- کرتے ہیں context inherit تک forks branch point دونوں
- کرتے ہیں diverge اس کے بعد و آزادانہ طور پر
- کرنے کے لیے مفید strategies test کرنے یا Approaches compare

کب شروع کریں session کا بجاؤ نئی Resume

- Tool results stale (files بدل چکی ہوں) ہوں
- ہو گیا ہو context degrade کافی وقت گزر چکا ہو اور
- کرنہ کر کے بجائے یہ بہتر ہے کہ "م نہ جو پایا اس کا مختصر خلاصہ یہ ہے : resume کر کے ساتھ tool data پرانے، کیا جائے restart ... " سد

باب 6: Prompt Engineering — Advanced Techniques

دستاویزات: [Prompt Engineering](#) | [Anthropic Cookbook](#)

6.1 Few-shot Prompting

Few-shot prompting میں input/output prompt 2–4 دکھانے کے لیے expected behavior کے طریقے جس میں شامل کیے جاتے ہیں examples

Few-shot text descriptions زیادہ مؤثر کیوں

- "وہ" کو کئی طریقوں سے سمجھا جا سکتا ہے precise جیسے "زیادہ" instruction میں
- Example decision logic اور expected format غیر میں طور پر
- Model pattern (صرف examples) کرنا generalize پر cases کو نئے

Few-shot examples کی اقسام اور استعمال

1. Ambiguous scenarios کے لیے examples:

```
Request: "My order is broken"
Action: Call get_customer -> lookup_order -> check status.
Rationale: "broken" damaged item آپ کو ہو سکتا ہے؛ order details میں درکار
```



```
Request: "Get me a manager"
Action: escalate_to_human call کریں فوراً
Rationale: Customer مانگ رہا ہے خود حل کرنے کی کوشش نہ کریں human واضح طور پر
```

2. Output formatting کے لیے examples:

```
Finding example:
{
  "location": "src/auth/login.ts:42",
  "issue": "SQL injection in the username parameter",
  "severity": "critical",
  "suggested_fix": "Use a parameterized query"
}
```

3. Acceptable problematic code کے لیے examples:

```
// Acceptable (flag نہ کریں):
const items = data.filter(x => x.active);

// Problem (flag کریں):
const items = data.filter(x => x.active == true); // Use strict equality ===
```

4. Different document formats سے extraction کے examples:

Document with inline citations:

"As shown in the study (Smith, 2023), the rate is 42%."

-> {"value": "42%", "source": "Smith, 2023", "type": "inline_citation"}

Document with bibliography references:

"The rate is 42%. [1]"

-> {"value": "42%", "source": "reference_1", "type": "bibliography"}

5. Informal measurements کے لیے examples:

Text: "about two handfuls of rice"

-> {"amount": "~100g", "original_text": "two handfuls", "precision": "approximate"}

Text: "a pinch of salt"

-> {"amount": "~1g", "original_text": "a pinch", "precision": "approximate"}

Few-shot purely rule-based instructions میں مؤثر کے جو non-standard measurement units اور informal خاص طور پر instructions کے لیے بہت متنوع ہوتی ہیں

Prompts استعمال strict JSON schemas کے لیے structured output جب format normalization rules میں: شامل کریں normalization rules میں prompt کریں

Normalization:

- Dates: ہمیشہ ISO 8601 (YYYY-MM-DD); "yesterday" -> absolute date compute کریں
- Currency: numeric amount + currency code; "five bucks" -> {"amount": 5, "currency": "USD"}
- Percentages: decimal fraction; "half" -> 0.5

وہ values inconsistent درست ہو مگر JSON syntactically کم ہوتے ہیں جہاں semantic errors اس سے

6.2 Explicit Criteria بمقابلہ Vague Instructions

Bad (vague):

Code comments کی accuracy چیک کریں

Conservative high-confidence findings report کریں - صرف

Good (explicit criteria):

Comment کو problematic میں اُس صورت میں flag کریں جب:

1. Comment کا actual code behavior سے CONTRADICT ہو
2. Comment کا حوالہ دہ variable یا function کسی غیر موجود
3. TODO/FIXME comment ایسے bug کو کسی

Flag نہ کریں:

- کہ لحاظ سے پرانہ ہوں style جو صرف comments ایسے
- wording inaccuracies معمولی
- Missing comments (الگ category یہ)

Severity criteria examples کے ساتھ define کے ساتھ:

CRITICAL: User کہ لیے runtime failure
Example: Payment process وقت NullPointerException

HIGH: Security vulnerability
Example: SQL injection, XSS, missing authorization checks

MEDIUM: Logic bug کہ بغیر فوری impact کہ
Example: Wrong sorting, off-by-one error

LOW: Code quality
Example: Duplication, کہ لیے data چھوٹہ suboptimal algorithm

6.3 Prompt Chaining

Prompt chaining میں توڑتا sequence کی focused steps کو ایک complex task

Step 1: auth.ts analyze کریں (صرف local issues)
-> Output: auth.ts میں issues کی list

Step 2: database.ts analyze کریں (صرف local issues)
-> Output: database.ts میں issues کی list

Step 3: Integration pass (cross-file dependencies)
-> Output: module boundaries پر issues

یہ کیوں اہم:

- **Attention dilution** — جب files دیکھتا تو کچھ files ایک ساتھ ہوت سی model سد بچاتا — جب bugs miss files دیکھتا
- سد سکتا shallow commentary سد سکتا اور باقی پر
- یقینی بناتا consistent analysis quality کہ لیے file ر
- Cross-file interactions الگ analysis کہ لیے الگ

Prompt chaining کب استعمال کریں dynamic decomposition بمقابلہ:

- **Prompt chaining** — predictable, repeatable tasks (code review, file migrations)
- **Dynamic decomposition** — open-ended investigations ان subtasks execution وتہ واضح وتہ

6.4 “Interview” Pattern

Solution implement Claude clarifying questions پوچھتا

Claude: "API کہ لیے caching implement چند سوالات
1. پسند کرتہ ہیں؟ cache invalidation strategy میں سد کون سی event-based یا TTL آپ
2. قابل قبول سد؟ stale data ہو تو کیا cache unavailable اگر
3. Caching per-user یا چاہیہ global؟
4. Cache کہ لیے expected data volume کہ کرنے

یہ کب مفید:

- Unfamiliar domain (fintech, healthcare, legal systems)
- Implications and tasks (cache strategies, failure modes)
- Viable approaches contextually, but selection is challenging

6.5 Validation اور Retry-with-Feedback

جب extracted data validation fail کرے:

```
Step 1: Document سے data extract کریں
Step 2: Validate کریں (Pydantic, JSON Schema, business rules)
Step 3: retry کریں کہ ساتھ context ہو error اگر:
- Original document
- Previous (incorrect) extraction
- Specific error: "Field 'total' = 150, but sum(line_items) = 145. Re-check values."
```

Retry مؤثر ہوگا:

- Format errors (date غلط format میں)
- Structural errors (field غلط جگہ)
- Arithmetic inconsistencies (model check دوبارہ کر سکتا ہے)

Retry گا کی مدد:

- Information source document میں موجود ہے
- Required context external (data کسی اور document میں کیا گیا)

Pydantic validation tool: Pydantic Python library جو schema-based data validation کے لیے استعمال ہوتی ہے۔ امتحان کے لیے اس کے نکات یہ ہیں:

- **Structural validation:** types, requiredness, enum constraints JSON کے بعد code میں جانچا جاتا ہے
- **Semantic validation:** custom validators business logic enforce کرتے ہیں (sum of items equals total! start_date < end_date)
- **Validate-retry loops:** Pydantic validation failure پر error message بنا کر Claude کو error context میں prompt ساتھ دوبارہ کریں
- **JSON Schema generation:** Pydantic models سے JSON Schema generate کر سکتے ہیں `tool_use` کے لیے۔ اس کا single source of truth ہے، جو ایک فراہم کرتا ہے

6.6 Self-correction

Internal contradictions detect کرنے کا pattern:

```
{
  "stated_total": "$150.00",
  "calculated_total": "$145.00",
  "conflict_detected": true,
  "line_items": [
    {"name": "Widget A", "price": 75.00},
    {"name": "Widget B", "price": 70.00}
  ]
}
```

آپ `conflict_detected` دونوں نکالتا ہے — اگر وہ مختلف ہوں تو Model stated value اور computed value کو discrepancy handle کرنے دیتا ہے

باب 7: Message Batches API

دستاویزات: [Message Batches](#)

7.1 Overview

کرنے دیتا ہے batches submit کے requests کے لیے asynchronous processing کو Message Batches API

Attribute	Value
Savings	50% کے مقابلے میں 50 synchronous calls
Processing window	24 hours (latency SLA guarantee) زیادہ سے زیادہ
Multi-turn tool calling	Supported نہیں (یک request = ایک response)
Correlation	request اور response کے لیے <code>custom_id</code> field کو جوڑنے کے لیے

7.2 Batch API کے مقابلے Synchronous API کے استعمال کریں

Task	API	Why
Pre-merge PR check	Synchronous	Developer 24 hours انتظار کر رہا ہے؛ قابل قبول نہیں
Overnight tech-debt report	Batch	savings % صبح تک چاہیے؛ Result 50
Weekly security audit	Batch	savings % فوری نہیں؛ 50
Interactive code review	Synchronous	درکار کے response فوری
10,000 documents process کرنا	Batch	Bulk processing؛ savings ہیں

7.3 custom_id کا استعمال

```
{
  "custom_id": "doc-invoice-2024-001",
  "params": {
    "model": "claude-sonnet-4-6",
    "max_tokens": 1024,
    "messages": [{"role": "user", "content": "Extract data from: ..."}]
  }
}
```

custom_id آپ کو یہ کرنے دیتا ہے:

- Result کو original document link سے
- Failure submit دوبارہ کی صورت میں صرف
- Successful documents process کو دوبارہ

7.4 Batch کو Failures میں Handle کرنا

1. 100 documents کا batch submit کریں
2. 95 succeed 5 fail (context limit exceeded) ہوں کریں
3. Failures کو custom_id سے identify کریں
4. Strategy (میں تقسیم کریں chunks کو long documents مثلاً) بدلیں
5. 5 failed documents submit کو دوبارہ صرف اُن

7.5 SLA Planning

تک لگ سکتا ہے 24 hours کو Batch API چاہے اور result میں hours اگر آپ کو 30

- Submission window: 30 - 24 = 6 hours
- Batches کو deadline 24 hours سے کم از کم
- Frequent submissions 4 hour windows میں تقسیم کریں

8: Task Decomposition Strategies

8.1 Fixed Pipelines (Prompt Chaining)

ہوتا ہے defined step سے:

Document -> Metadata extraction -> Data extraction -> Validation -> Enrichment -> Final output

کب استعمال کریں:

- Task structure predictable (reviews ایک template follow کریں)
- پلاٹ سڈ معلوم ہوں steps تمام
- چا پیڈ reproducibility اور stability آپ کو

8.2 Dynamic Adaptive Decomposition

Subtasks intermediate results پر بنیاد کی generate ہیں:

1. "Legacy codebase کے لیے tests add کریں"
2. -> structure map (Glob, Grep) کریں
3. -> 3 modules بغیر tests، partial coverage کے ساتھ
4. -> Prioritize: payments module شروع کریں (high risk)
5. -> دریافت ہوئی dependency پر external API: کام کے دوران
6. -> Adapt: tests کے لیے external API لکھنے سے پہلے mock add کریں

کب استعمال کریں:

- Open-ended investigative tasks
- شروع میں معلوم نہ ہو scope جب مکمل
- پر منحصر ہو results کے step پچھلے step جب ر

8.3 Multi-pass Code Review

10+ files والی pull requests کے لیے:

```
Pass 1 (per-file): auth.ts analyze کریں -> local issues کی list
Pass 1 (per-file): database.ts analyze کریں -> local issues کی list
Pass 1 (per-file): routes.ts analyze کریں -> local issues کی list
...
Pass 2 (integration): files درمیان کے relationships analyze کریں
-> Cross-file issues: inconsistent types, circular dependencies
```

14 files کے لیے pass پر ایک کی خراب:

- Attention dilution: shallow کچھ کی، deep analysis کی files کچھ
- Inconsistent comments: approve دوسری میں، pattern flag میں ایک file
- Missed bugs: cognitive overload واضح سڈ errors کی وجہ سے

9 Escalation اور Human-in-the-Loop: باب 9

9.1 Escalate کی طرف Human کب

Escalation triggers (rules واضح):

Situation	Action
-----------	--------

Customer "get me a manager" واضح طور پر کہتا ہے	کریں؛ حل کرنے کی کوشش نہ کریں escalate فوراً
Policy request کو cover نہ کرتی ہو	Escalate policy جب competitor price matching مثلاً کریں (ہو)
Agent پیش رفت نہ کر سکے	کریں escalate کے بعد attempts مناسب تعداد میں
Threshold financial operation سے اوپر	Escalate کے ذریعے prompt enforce، کے ذریعے hook ترجیحاً کریں (نہیں)
Customer multiple matches تلاش کرتے وقت	مانگیں؛ انداز نہ لگائیں identifiers مزید

نہیں ہیں reliable trigger جو:

Unreliable method	Why it fails
Sentiment analysis	نہیں کرتا correlate سے Customer کا mood case complexity
Model self-rated confidence (1–10)	کمزور calibration غلط ہو سکتا ہے؛ Model confidently
Automatic classifier	موجود نہ ہو training data؛ ممکن Overengineering

9.2 Escalation Patterns

Immediate escalation:

Customer: "I want to speak to a manager"
 Agent: [کرتا escalate_to_human call فوراً]
 NOT: "I can help with your issue, let me..."

Resolve escalation: کی کوشش کے بعد

Customer: "My refrigerator broke two days after purchase"
 Agent: [کرتا warranty replacement offer، چیک کرتا]
 escalate -> مطمئن نہ ہو customer اگر

Nuanced escalation (acknowledge → resolve → reiteration پر escalate):

Customer: "This is outrageous, I'm very unhappy with the quality!"
 Agent: [frustration acknowledge] "سمجھتا ہوں frustration میں آپ کی"
 [resolution offer] "replacement یا refund ہو سکتا ہے میں"
 Customer: "No, I want to talk to someone!"
 Agent: [customer insist دوبارہ -> immediate escalation]

کریں escalate دیں، اور صرف اسی وقت concrete solution کریں، پھر emotion acknowledge بنیادی اصول: پہلے
 manager (نہیں) کریں escalate کے پہلے اطلاع پر dissatisfaction کی خواہش دہرائیں human customer جب
 (مانگندہ کے برابر نہیں ہیں)

Policy gap کے لیے escalation:

Customer: "Competitor X has this item 30% cheaper—give me a discount"
Policy: کرتی ہے price adjustments cover پر site صرف اپنی
Agent: [escalate – policy competitor price matching کو cover نہیں کرتی]

9.3 Structured Handoff Protocols

Escalation کو دینی چاہیے structured summary human agent پر:

```
{
  "customer_id": "CUST-12345",
  "customer_name": "Ivan Petrov",
  "issue_summary": "Refund request for a damaged item",
  "order_id": "ORD-67890",
  "root_cause": "Item arrived damaged; photos attached",
  "actions_taken": [
    "Verified customer via get_customer",
    "Confirmed order via lookup_order",
    "Offered a standard replacement – customer insists on a refund"
  ],
  "refund_amount": "$89.99",
  "recommended_action": "Approve a full refund",
  "escalation_reason": "Customer requested to speak with a manager"
}
```

دیکھتا ہے summary تک رسائی نہیں ہوتی — صرف یہ full conversation transcript کو Human operator کو دینی چاہیے self-contained اس لیے یہ مکمل اور

9.4 Confidence Calibration اور Human Oversight

Data extraction systems کے لیے:

1. **Field-level confidence scores:** model کے لیے extracted field کا confidence score نکالتا ہے
2. **Calibration:** labeled validation sets کے thresholds tune کریں استعمال کر کے
3. **Routing:**
 - High confidence + stable accuracy -> automated processing
 - Low confidence یا ambiguous sources -> human review

Stratified random sampling:

- High-confidence extractions کا sample audit بھی باقاعدگی سے کریں
- Aggregate 97% accuracy کا document type کسی مخصوص 40% errors کے لیے
- Accuracy کا analyze کے حساب سے field اور document type، میں overall کو صرف

Multi-agent Systems میں Error Handling: باب 10

10.1 Error Categories

Category	Examples	Retryable	Agent action
Transient	Timeout, 503, network failure	Yes	Exponential backoff ساتھ retry کریں
Validation	Invalid input format, missing required field	No (input fix کریں)	Request modify اور retry کریں
Business	Policy violation, threshold exceeded	No	پیش alternative کو سمجھائیں؛ User کریں
Permission	Access denied	No	Escalate کریں

10.2 Error-handling Anti-patterns

Anti-pattern	Problem	Correct approach
Generic status “search unavailable”	Coordinator decide نہیں کر سکتا کہ recover کیسے کرنا ہے	Error type, query, partial results, alternatives واپس کریں
Silent suppression (empty result = success)	Coordinator match سمجھتا ہے کہ failure ملا، حالانکہ	“no results” اور “search failure” میں فرق کریں
Workflow پر پورا failure ایک کرنا abort	ضائع ہو جاتا ہے partial results تمام	Partial results ساتھ جاری رکھیں؛ gaps annotate کریں
Subagent میں infinite retries	Latency اور resources کا ضیاع	Local recovery (1–2 retries)، پھر coordinator تک propagate

10.3 Structured Subagent Error

```
{
  "status": "partial_failure",
  "failure_type": "timeout",
  "attempted_query": "AI impact on music industry 2024",
  "partial_results": [
    {"title": "AI Music Generation Report", "url": "...", "relevance": 0.8}
  ],
  "alternative_approaches": [
    "Try a narrower query: 'AI music composition tools'",
    "Use an alternative data source"
  ],
  "coverage_impact": "Not covered: AI impact on music production"
}
```

کو فیصلہ کرنا کہ آیا ضروری معلومات دیتا ہے coordinator

- Modified query ساتھ retry کریں؟
- Partial results استعمال کریں؟
- subagent کو delegate کسی اور کریں؟
- gap annotate کریں؟ بغیر جاری رکھیں اور section اس

10.4 Final Synthesis میں Coverage Annotations

Report: AI Impact on Creative Industries

Visual Art (FULL COVERAGE)

[research results]

Music (PARTIAL COVERAGE – search agent timeout)

[partial results]

⚠️ Note: coverage search agent timeout سے محدود ہے اس section کی

Literature (FULL COVERAGE)

[research results]

Context میں Production Systems: باب 11

Management

کریں Extract میں Separate Block کو Facts 11.1

Conversations history کو (جو summarize کے دوران summarize ہو جاتی ہے) key facts کو extract میں structured block میں:

```
=== CASE FACTS (کیا جائے update آئے fact جب بھی نیا) ===
Customer ID: CUST-12345
Order ID: ORD-67890
Order Date: 2025-01-15
Order Amount: $89.99
Issue: Damaged item on delivery
Customer Request: Full refund
Status: Pending manager approval
===
```

کیوں نہ ہو summarize کتنی ہی history میں شامل کریں، چاہے prompt میں block نہ ہو

کریں Trimming کو Tool Results 11.2

اگر `lookup_order` 40+ fields مگر current task میں صرف 5 درکار ہیں واپس کرتا ہے مگر

```
# PostToolUse hook: صرف relevant fields رکھیں
@hook("PostToolUse", tool="lookup_order")
def trim_order_fields(result):
    return {
        "order_id": result["order_id"],
        "status": result["status"],
        "total": result["total"],
        "items": result["items"],
        "return_eligible": result["return_eligible"]
    }
```

کم ہوتی ہے noise بچتا ہے اور context اس سے

Position-aware Input 11.3

Lost-in-the-middle effect کو critical information کو ذہن میں رکھتے ہوئے

3 critical vulnerabilities ... ملین

```
=== File auth.ts ===
```

■ ■ ■

```
=== File database.ts ===
```

• • •

Priority: merge `auth.ts` vulnerabilities fix کریں.

Long investigations میں agent key findings scratchpad file □□ لکھ سکتا

```
# investigation-scratchpad.md
## Key findings
- PaymentProcessor in src/payments/processor.ts inherits from BaseProcessor
- refund() تین places سے call ہوتی ہے: OrderController, AdminPanel, CronJob
- External PaymentGateway API limit: 100 req/min
- Migration #47 نے refund_reason (NOT NULL) add کیا - 01-12-2024
```

scratchpad دوبار چلائے گا agent discovery (میں session یا نئی) کو جائے context degrade جب consult کر سکتا ہے

```
Main agent: "payments module کی dependencies investigate کریں"
  -> Subagent (Explore): 15 files read کرتا, imports trace کرتا
  -> Returns: "Payments depends on AuthService, OrderModel, and the external
PaymentGateway API"
```

Main agent: context 15 میں files ایک بحالہ line ۰۰ کہتا

Separate context layer: Multi-agent systems میں کام کرتا ہے — context budget محدود subagent میں — صرف اپنے task اس کے لیے ضروری information کو ملتی ہے۔ Coordinator ایک separate context layer طور پر context allocate کرتا ہے، اور global state store، کرتا ہے subagent outputs aggregate کرتا ہے۔ وہ اس کے agent window میں نہ دیتا ہے، جہاں ایک “context leakage” ہے۔ اس کے دوسروں کے لیے غیر متعلق ہے۔

Subagents $\square_{\downarrow}$ $\square_{\leq}$ constrained context budgets:

- Minimal context **بہجیں**: specific task + ضروری data
- Subagent کو raw data dumps **بجائے** structured results کی **ہدایت** دیں
- `allowedTools` **subagent** کو toolset کو **کم** — کم distractions کا مطلب کم tools کریں — **context cost**

11.6 Structured State Persistence (crash recovery لی کی)

کرتا export پر location ایک معلوم state اپنی agent ر:

```
// agent-state/web-search-agent.json
{
  "status": "completed",
  "queries_executed": ["AI music 2024", "AI music composition"],
  "results_count": 12,
  "key_findings": [...],
  "coverage": ["music composition", "music production"],
  "gaps": ["music distribution", "music licensing"]
}
```

کرتا manifest load پر ایک Coordinator resume

```
// agent-state/manifest.json
{
  "web-search": "completed",
  "doc-analysis": "in_progress",
  "synthesis": "not_started"
}
```

12 کو محفوظ رکھنا Provenance: باب 12

12.1 Attribution Loss Problem

گم ہو سکتا link "claim → source" کی جاتی ہیں، تو results summarize سے sources جب متعدد

Bad: "The AI music market is estimated at \$3.2B." (No source, no year.)

Good:

```
{
  "claim": "The AI music market is estimated at $3.2B.",
  "source_url": "https://example.com/report",
  "source_name": "Global AI Music Report 2024",
  "publication_date": "2024-06-15",
  "confidence": 0.9
}
```

12.2 Conflicting Data کو Handle کرنا

دیں values مختلف sources جب دو

```
{
  "claim": "Share of AI-generated music on streaming platforms",
  "values": [
    {
      "value": "12%",
      "source": "Spotify Annual Report 2024",
      "date": "2024-03",
      "methodology": "Automated classification"
    },
    {
      "value": "8%",
      "source": "Music Industry Association Survey",
      "date": "2024-07",
      "methodology": "Survey of 500 labels"
    }
  ],
  "conflict_detected": true,
  "possible_explanation": "Methodology اور time period فرق کا"
}
```

کو coordinator کے ساتھ محفوظ رکھیں اور attribution کو اندھا دھند منتخب نہ کریں۔ دونوں کو value کسی ایک فیصلہ کرنے دیں۔

12.3 Correct Interpretation کے لیے Dates شامل کریں

سمجھ لیا جا سکتا ہے contradiction کو temporal differences کے بغیر Dates:

Bad: "Source A says 10%, source B says 15%. Contradiction."

Good: "Source A (2023) says 10%, source B (2024) says 15%. Likely 5+ سال میں 1% growth."

12.4 Content Type کے مطابق Render کریں

میں نہ ٹھونسیں format کے چیز کو ایک کی:

- Financial data -> tables
- News اور analysis -> prose
- Technical findings -> structured lists
- Time series -> chronological ordering

باب 13: Claude Code کے Built-in Tools

13.1 Tool Selection Reference

Task	Tool	Example
تلاش کرنا files کے نام/pattern	Glob	<code>**/*.test.tsx</code> , <code>src/components/**/*.ts</code>
File contents کے اندر search کرنا	Grep	Function name, error message, import
File مکمل پڑھنا	Read	Analysis کے لیے file load کرنا
بنانا file نیا	Write	Scratch کے file create کرنا
precise edit میں file موجود	Edit	Unique text match کے ذریعے specific snippet replace کرنا
Shell command چلانا	Bash	git, npm, tests, build

13.2 Incremental Investigation Strategy

نہ: پڑھیں کے سمجھ بوجھ کو رفتہ رفتہ بنائیں files ایک ساتھ تمام

1. Grep: entry points تلاش کریں (function definition, export)
2. Read: پڑھیں files ملنے والی
3. Grep: usages تلاش کریں (import, calls)
4. Read: consumer files پڑھیں
5. کریں repeat نہ مل جائے picture جب تک مکمل

13.3 Fallback: Edit کے بجائے Read + Write

و: و جائے fail کی وجہ سے Edit non-unique text match جب

1. Read — لوڈ کریں file content مکمل
2. Content کو programmatically modify کریں
3. Write — لکھ دیں updated version

امتحانی ڈومین نوٹس: II حصہ

1 Agent Architecture اور Orchestration ڈومین: (27%)

1.1 Autonomous Task Execution لیکیلیکی Agentic Loops Design کرنا

Key knowledge:

- Agent loop lifecycle: Claude request بھیجیں، `stop_reason` ("tool_use" بمقابلہ "end_turn") چیک واپس کریں results کیلیکی iteration لیکیلیکی، اگلی tools execute کریں
- Tool results conversation history میں append کیلیکی تاکہ لیکیلیکی model اگلا action کر سکے
- Model-driven decision making (Claude اگلا tool لیکیلیکی چنتا لیکیلیکی) بمقابلہ hard-coded decision trees

Key skills:

- Flow control: جب `stop_reason = "tool_use"` تو loop جاری رکھیں اور `"end_turn"` پر رک جائیں
- Iterations کرنا append میں context کو tool results کیلیکی درمیان
- Anti-patterns لیکیلیکی completion لیکیلیکی assistant text parse کرنا، primary stopping mechanism طور لیکیلیکی، کرنا arbitrary iteration limits استعمال

1.2 Multi-agent Systems (Coordinator–Subagent) Orchestrate کرنا

Key knowledge:

- Hub-and-spoke architecture: coordinator تمام inter-agent communication، error handling، اور routing کا مالک ہوتا لیکیلیکی
- Subagents isolated context لیکیلیکی خود بخود لیکیلیکی — لیکیلیکی coordinator کی history inherit لیکیلیکی میں کام کرتے لیکیلیکی ہیں
- Coordinator responsibilities: task decomposition، delegation، result aggregation، dynamic selection of subagents
- Coordinator کی overly narrow decomposition کا خطرہ لیکیلیکی

Key skills:

- Research coverage کم لیکیلیکی duplication کیلیکی درمیان تقسیم کریں تاکہ لیکیلیکی subagents کو
- Iterative refinement loops implement کریں (coordinator synthesis evaluate اور tasks re-route لیکیلیکی کر لیکیلیکی)
- Observability تمام لیکیلیکی لیکیلیکی route لیکیلیکی ذریعہ لیکیلیکی coordinator communication لیکیلیکی لیکیلیکی

1.3 Subagent Calls, Context Passing, اور Spawning Configure کرنا

Key knowledge:

- `Task` tool subagents spawn؛ coordinator کو `allowedTools` میں "Task" ہونا چاہیے؛
- Subagent context prompt میں explicitly؛ ہونا چاہیے؛
- `AgentDefinition` configuration: descriptions, system prompts, tool constraints
- Alternatives explore کرنا کے لیے `fork_session` ذریعہ session management

Key skills:

- Previous agents کے full outputs subagent prompt میں شامل کریں
- Context pass کرنا کے لیے separate `data` اور `metadata` کو وقت کرتے
- `Task` calls کے parallel subagents spawn میں متعدد coordinator turn ایک
- Coordinator prompts goals اور quality criteria کے طور پر لکھیں، step-by-step instructions کے طور پر نہیں

1.4 Enforcement اور Handoff Patterns ساتھ Multi-step Workflows Implement کرنا

Key knowledge:

- Workflow ordering کے لیے `programmatic enforcement` (hooks, preconditions) اور `prompt guidance` کے درمیان فرق
- `deterministic guarantees` (مثلاً مالی operations، `identity verification`)، `prompts` درکار ہوں، کافی نہیں ہوتے
- Escalation کے دوران `structured handoff protocols` (customer ID, reason, recommended action)

Key skills:

- مکمل نہ ہونے والے steps کریں جب تک `block` کے downstream calls جو `programmatic preconditions` ایسے (مثلاً `get_customer` کے verified ID تک واپس کرنا، `process_refund` block کریں)
- Multi-aspect customer requests items کو الگ میں تقسیم کریں
- Human کو escalate کرتے وقت `structured summaries` produce کریں

1.5 Tool Calls Intercept اور Data Normalize کرنا کے لیے Agent SDK Hooks

Key knowledge:

- Hook patterns (مثلاً `PostToolUse`) tool results کو model کے consume کے لیے intercept کرنا
- Hooks outgoing calls intercept کے لیے `compliance rules` enforce کرنا (مثلاً `threshold` اوپر `refunds` block کرنا)
- Hooks `deterministic guarantees` دیتی ہیں، جبکہ `probabilistic compliance` prompt instructions دیتی ہیں

Key skills:

- `PostToolUse` hooks سے data formats normalize کریں (Unix timestamps, ISO 8601, numeric status codes)
- Policy-violating actions block کرنے کے لیے interception hooks استعمال کریں اور escalation کی طرف redirect کریں
- منتخب کریں hooks کے بجائے prompts چاہیے۔ تو تو guaranteed compliance کو business rules جب

1.6 Complex Workflows کے Task Decomposition Strategies

Key knowledge:

- **Fixed pipelines** (prompt chaining) کے مقابلے میں **dynamic adaptive decomposition** کی بنیاد پر intermediate results
- Prompt chaining: sequential steps (پھر integration pass، analyze کو الگ سے)
- Adaptive investigation plans جو discovered information کی بنیاد پر subtasks generate کرتے ہیں

Key skills:

- Predictable multi-aspect reviews کے لیے prompt chaining؛ open-ended investigations کے لیے dynamic decomposition
- Large code reviews کو per-file analysis + separate cross-file integration pass میں تقسیم کریں
- Open-ended tasks کو degrade ہونے سے پہلے prioritized plan کریں، پھر structure map کریں: پھر

1.7 Session State, Resuming, اور Forking

Key knowledge:

- Named sessions continue کرنے کے لیے `--resume <session-name>`
- Shared context سے independent investigation branches بنانے کے لیے `fork_session`
- Resume کی اہمیت file changes کو agent کرتے وقت
- Structured summary کے ساتھ session، stale results کے ساتھ resuming زیادہ reliable سے زیادہ

Key skills:

- Named investigation sessions continue کرنے کے لیے `--resume` استعمال کریں
- Approaches parallel compare کرنے کے لیے `fork_session` استعمال کریں
- Resuming (context ابھی current ہے) session شروع کرنے اور نئی (results stale ہیں) درمیان انتخاب کریں

5.10 fork_session اور Session Management

`--resume <session-name>` named session کو resume کرتا ہے:

```
claude --resume investigation-auth-bug
```

- جاری رکھتا conversation کے ساتھ پچھلی saved context
- پر پھیلی ہوں ان کے لیے مفید sessions جو متعدد Long investigations
- Risk: tool results stale بدل چکی ہوں تو files کے بعد session اگر پچھلی

fork_session shared context سے independent branch بنانا:

```
Codebase investigation
|
fork_session
/      \
Approach A:  Approach B:
Redux       Context API
```

- کرتے ہیں context inherit تک forks branch point دونوں
- کرتے ہیں diverge اس کے بعد وہ آزادانہ طور پر
- کرنے کے لیے مفید strategies test کرنے یا Approaches compare

Resume نئی session کے بجائے نئی:

- Tool results stale ہوں (files بدل چکی ہوں)
- وہ گیا context degrade کافی وقت گزر چکا ہو اور
- کرنے کے بجائے یہ بہتر ہے کہ "م نہ جو پایا اس کا مختصر خلاصہ یہ ہے": resume کے ساتھ tool data پرانے کیا جائے restart "... سے"

2 Tool Design اور MCP Integration (18%) ڈومین

2.1 Clear Descriptions کے ساتھ Tool Interfaces Design کرنا

Key knowledge:

- Tool descriptions minimal descriptions کرنا؛ LLM tools select میں جن سے mechanism و بنیادی unreliable selection دیتی
- Input formats, example queries, edge cases, اور applicability boundaries کی اہمیت شامل کرنے
- Ambiguous یا overlapping descriptions misrouting کا سبب بنتی ہیں
- System prompt wording tools کے associations کے ساتھ غیر ارادی بنا سکتی ہیں

Key skills:

- کریں distinguish سے واضح طور پر similar alternatives کو tool لکھیں جو descriptions ایسی
- Functional overlap کے لیے (مثلاً `analyze_content` -> `extract_web_results`) کریں tools rename ختم کرنے کے لیے
- واضح ہوں input/output contracts میں تقسیم کریں جن کے specialized tools کو General-purpose tools

2.2 MCP Tools کے لیے Structured Error Responses Implement کرنا

Key knowledge:

- MCP tool responses میں `isError` flag
- Transient errors** (timeouts), **validation errors** (bad input), **business errors** (policy violations), اور **access/permission errors** کے درمیان فرق
- Generic errors ("Operation failed") میں recovery decisions روک دینے کے لیے
- Retryable اور non-retryable errors میں فرق

Key skills:

- `errorCategory` (transient/validation/permission), `isRetryable` جیسی human-readable message اور `retryable: false` کے ساتھ user-facing explanations کے لیے واضح structured metadata واپس کریں
- Business-rule violations کے لیے واضح `retryable: false` استعمال کریں
- Transient failures کے لیے subagents میں local recovery کے لیے صرف وہ errors propagate کریں جو خود حل نہ کر سکیں
- Access failures (retry decision) اور valid empty results (no matches) میں فرق کریں

2.3 Agents میں Tools Allocation اور `tool_choice` Configure کرنا

Key knowledge:

- کم کرتی tool selection reliability (مثلاً 18 کے بجائے 4-5) tools کے زیادہ
- Specialization کے لیے agents رکھنے والے tools سے باہر
- Scoped tool access: role-relevant tools + cross-role utilities
- `tool_choice: "auto"`, `"any"` اور forced tool selection (`{"type": "tool", "name": "..."}`)

Key skills:

- تک محدود رکھیں relevant tools صرف subagent کا toolset
- General tools کو constrained alternatives کے لیے بدلیں (مثلاً `fetch_url` -> `load_document`)
- `tool_choice: "any"` کے بجائے text answer کے لیے استعمال کریں تاکہ
- Specific tool force کے لیے execution order assured کریں تاکہ

2.4 MCP Servers اور Claude Code میں Agent Workflows Integrate کرنا

Key knowledge:

- MCP server scope: user کے لیے experiments کے مقابلے میں project (`.mcp.json`) کے لیے (`~/claude.json`)
- Secrets management کے لیے `.mcp.json` میں environment variable substitution (مثلاً `${GITHUB_TOKEN}`)

- available ہوتے ہیں اور ایک ساتھ discover پر tools connection کے connected MCP servers کے
- MCP resources بطور "content catalogs" (task summaries, database schemas) exploratory tool calls کم کرتے ہیں

Key skills:

- Shared MCP servers کو project میں `.mcp.json` env-var-based tokens ساتھ configure کے
- Personal/experimental servers کو `~/claude.json` میں رکھیں
- Standard integrations کے لیے community MCP servers کو custom servers پر ترجیح دیں

2.5 Built-in Tools (Read, Write, Edit, Bash, Grep, Glob) منتخب اور کرنا Apply

Key knowledge:

- **Grep**: file contents کے اندر search (function names, error messages, imports)
- **Glob**: name/extension patterns کے تلاش کرنا files کے
- **Read/Write**: full-file operations؛ **Edit**: unique text matches کے ذریعے precise changes
- **Read + Write** پر واپس جائیں تو fail کی وجہ سے کے Edit non-unique matches کے

Key skills:

- Content search کے لیے Grep اور pattern-based discovery کے لیے
- **Read** کے لیے flows trace پھر Grep کے لیے entry points: سمجھ بوجھ رفتہ رفتہ بنائیں
- Wrapper modules کے ذریعے function usage trace کریں

3 اور Claude Code Configuration: ڈومین Workflows (20%)

3.1 Hierarchy, Scope, اور Modular Organization کے ساتھ CLAUDE.md Configure کرنا

Key knowledge:

- CLAUDE.md hierarchy: user (`~/claude/CLAUDE.md`), project (`.claude/CLAUDE.md` یا root `CLAUDE.md`), اور directory-level (subdirectories میں CLAUDE.md)
- User-level settings اور user صرف ایک کے لیے VCS کے ذریعے share کے لیے اور VCS پر لاگو ہوتے ہیں
- External files refer کے لیے `@path` syntax (مثلاً `@./standards/coding-style.md`) تاکہ CLAUDE.md modular بن سکے
- Monolithic CLAUDE.md کے بجائے topic-focused rule files کے لیے `.claude/rules/` directory

Key skills:

- user-اس لیے میں ملتی ہیں کیونکہ میں وہ instructions کو team member (نہیں) کریں Hierarchy issues diagnose level میں ہیں، project-level میں نہیں
- (مثلاً) @path کرنے کے لیے standards selectively include میں CLAUDE.md کے package میں استعمال کریں (@./standards/testing.md)
- files (testing.md, api-conventions.md, deployment.md) .claude/rules/ کو متعدد CLAUDE.md بڑے میں تقسیم کریں

3.2 Custom Slash Commands اور Skills بنانا اور Configure کرنا

Key knowledge:

- `~/.claude/commands/` بمقابلہ `shared` (VCS ذریعہ) میں **Project commands** میں `~/.claude/commands/`
- `~/.claude/skills/` میں SKILL.md frontmatter: `context: fork` , `allowed-tools` , `argument-hint`
- `context: fork` skill کو isolated subagent context میں چلاتا ہے تاکہ main session pollute نہ ہو
- Personal skill variants `~/.claude/skills/` میں رکھے جاسکتے ہیں

Key skills:

- Project slash commands `.claude/commands/` پوری team میں رکھیں تاکہ پوری
- Verbose output `context: fork` استعمال کریں `skills isolate` والے `کے لیے`
- Skill `allowed-tools` محدود کرنے کے لیے `tools` استعمال کر کے استعمال کریں
- Required parameters `argument-hint` مانگنے کے لیے استعمال کریں

3.3 Conditional Convention Loading لیکھیں Path-specific Rules استعمال کرنا

Key knowledge:

- `.claude/rules/` files میں YAML frontmatter `paths` تاکہ `glob patterns` شامل ہو سکتا ہے۔ `rules activate` ہوں
- Path-scoped rules `matching files edit` وقت `load` کریں، `tokens` اور `context`، `بچتے` ہیں
- Glob-based path rules `directory-level CLAUDE.md` میں جب `conventions` سے `بتر` ہو سکتے ہیں جب `tests` (مثلاً) `apply` میں `directories`

Key skills:

- load matching files بنائیں تاکہ صرف `.claude/rules/` کے ساتھ `paths: ["terraform/**/*"]` ہوں
- Glob patterns (`**/*.test.tsx`) کے لحاظ سے file type کا قطع نظر location استعمال کریں تاکہ conventions apply ہوں
- path-specific کے بجائے `CLAUDE.md` directory-level بھر میں پھیلی ہوں تو conventions codebase جب کو ترجیح دیں

Key skills:

- Interactive input پر hang لے لے کے لیجئے کہ وہ CI میں Claude Code کو -p کے ساتھ چلائیں
- Structured results (مثلاً inline PR comments) کے لیے --output-format json + --json-schema استعمال کریں
- new/unfixed issues (صرف) شامل کریں review results کے وقت پچھلے run کے بعد دوبارہ commits نہ (کریں report)
- Test generation quality کے لیے CLAUSE.md میں testing standards اور available fixtures document کریں
- style نہ ہو اور duplication میں شامل کریں تاکہ context کو existing test files کے وقت tests generate نہ کرے consistent ہو

4 Prompt Engineering اور Structured Output (20%)

4.1 Accuracy کے Prompts کے ساتھ Explicit Criteria، تر کرنے کے لیے لی کرنا Design

Key knowledge:

- Explicit criteria vague instructions میں زیادہ مؤثر ہوتے ہیں (مثلاً "flag comments only when they contradict code" بمقابلہ "check comment accuracy")
- "be more conservative" generic guidance concrete categorical criteria کے مؤثر ہیں
- False positives developer trust پر اثر انداز ہوتے ہیں: کچھ categories میں high false-positive rates درست کم کر دیتی ہیں trust پر بھی categories

Key skills:

- Review criteria define کرنا (bugs, security) اور ignore کرنا (minor style) report کرنا
- High false-positive rates والی categories طور پر disable عارضی
- explicit severity criteria define کے ساتھ code examples کے لیے level پر

4.2 Output Consistency کے Few-shot Prompting، تر کرنے کے لیے لی کرنا استعمال کرنا

Key knowledge:

- Few-shot examples consistently formatted, actionable output مؤثر ہے کہ سب کے method پیدا کرنے کا
- Few-shot cases (tool selection, test coverage gaps) کو دکھا سکتا ہے
- Few-shot model میں defaults کے میں مدد دیتا ہے، صرف generalize پر patterns کو نہ
- Few-shot extraction tasks میں hallucinations کو کر سکتا ہے

Key skills:

- Ambiguous scenarios کے لیے rationale 4–2 کے ساتھ targeted examples دینے
- Output format (location, issue, severity, suggested fix) کے ساتھ examples دکھانے والے
- میں فرق دکھائیں real issues اور acceptable code patterns دینے جو examples ایسے
- دینے examples کے extraction سے درست documents والے structures مختلف

4.3 Structured Output کے ساتھ JSON Schemas اور tool_use کرنا Enforce

Key knowledge:

- tool_use with JSON Schemas schema-conformant output اور JSON syntax errors کی ضمانت دینے کا طریقہ قابل اعتماد کرنا کا سب سے
- tool_choice: "any" واپس کر سکتا ہے؛ model text میں "any" منتخب کرتی ہے؛ forced selection specific tool؛
- Strict JSON Schemas syntax errors semantic errors ختم کرتی ہیں مگر (توٹاں؛ totals) میں غلط fields (میں غلط values)
- Schema design: required vs optional fields؛ enums کے ساتھ "other" + detail string extensibility کے لیے

Key skills:

- JSON Schemas کے ساتھ extraction tools define کریں اور tool_use results سے data parse کریں
- استعمال کریں tool_choice: "any" یقینی بنانے کے لیے structured output کے ساتھ تو schemas متعدد
- Specific tool call force کریں: tool_choice: {"type": "tool", "name": "extract_metadata"}
- Source میں information کی صورت میں values fabricate نہ ہونے کی صورت میں optional/nullable رکھیں
- Extensible categorization کے لیے "unclear" اور "other" enum values + detail fields جیسے استعمال کریں

4.4 Extraction Quality کے لیے Validation, Retries, Feedback اور Loops Implement کرنا

Key knowledge:

- Retry-with-error-feedback: correction guide کے لیے concrete validation errors میں retry prompt کرنے کے لیے شامل کریں
- فوائد ہونے میں retries کے لیے information source سے تو واپس کرنا
- Feedback loop design: finding کو trigger والا pattern (detected_pattern) track کریں
- Semantic errors (totals reconcile نہ ہونے کے لیے) syntax errors (جو tool_use سے address ہونے کے لیے)

Key skills:

- Original document, incorrect extraction, اور specific validation errors کے ساتھ follow-up prompts دینے
- تو اس کی شناخت کریں (میں سے واپس کرنا external document صرف required info کے لیے) واپس کرنا

- False positives analyze `detected_pattern` fields میں findings کرنے کے لیے
- Discrepancies detect `calculated_total` اور `stated_total` extract کرنے کے لیے self-correction design کریں

4.5 Efficient Batch Processing Strategies Design کرنا

Key knowledge:

- Message Batches API: 50% savings, 24-hour processing window, latency SLA guarantee
- Batch processing non-blocking tasks (overnight reports, audits) اور blocking tasks (pre-merge checks) کے لیے مناسب
- Batch API single request میں multi-turn tool calling support
- `custom_id` fields batch request/response correlate کرتی ہیں

Key skills:

- Blocking checks synchronous API اور overnight/weekly workloads کے لیے استعمال
- SLA needs مطابق batch submission cadence plan کریں (24-hour guarantee مثلاً 30 processing 4-hour windows کے ساتھ)
- Failed documents (identified by `custom_id`) submit کر کے failures handle
- Large-scale processing کے ساتھ sample prompts iterate

4.6 Multi-instance اور Multi-pass Review Architectures Design کرنا

Key knowledge:

- Self-review limitations: model reasoning context کو فیصلوں اور اپنے challenge ساتھ رکھتا ہے اور اپنی امکان کم ہوتا ہے
- Independent review instances (generation context بغیر) subtle issues میں بہتر ہوتی ہیں
- Multi-pass review: per-file local analysis + cross-file integration pass attention dilution سے بچانا

Key skills:

- Changes review second independent Claude instance کے بغیر generation context کرنے کے لیے
- Multi-file reviews کو per-file passes + integration passes میں تقسیم کریں تاکہ cross-file dataflow analysis ہو سکے
- Self-rated confidence کے ساتھ verification passes کے reviews استعمال کر کے calibrated سے طریقہ

5 Reliability اور Context Management ڈومین: 5 (15%)

5.1 Critical Information Conversation Context محفوظ رکھنا اور Critical Information Manage کرنا

Key knowledge:

- Progressive summarization کے خطرات: numeric values, percentages, اور dates vague summaries میں بدل جاتے ہیں
- Lost-in-the-middle effect: models long inputs کے شروع اور آخر کو زیادہ reliable سے کم کر سکتے ہیں، لیکن درمیان کے findings miss ہیں
- Tool outputs relevance میں context کے مقابلہ میں disproportionately کم ہو سکتے ہیں (جب 5+ fields 40+ جمع ہو سکتے ہیں)
- Subsequent API requests میں full conversation history کی اہمیت

Key skills:

- Transactional facts کو summarized history کے ساتھ persistent “case facts” block میں نکالیں
- Verbose tool outputs کو relevant fields تک trim کریں
- Key findings کو aggregated data کے شروع میں explicit section headings رکھیں
- Subagents کے structured outputs میں metadata (dates, sources) شامل کروائیں

5.2 Effective Escalation Patterns Design اور Ambiguity Resolve کرنا

Key knowledge:

- Suitable escalation triggers: human کی explicit request, policy gaps/exceptions, اور پانا
- Immediate escalation (explicit request) کے اندر attempt-to-resolve (agent scope) کے مقابلہ میں
- Sentiment analysis اور model confidence self-ratings case complexity کے unreliable proxies ہیں
- Multiple customer matches کے heuristic guessing کے بجائے additional identifiers چائیں

Key skills:

- System prompt میں explicit escalation criteria few-shot examples کے ساتھ دیں
- Human کی explicit request کو بغیر اضافی investigation کے فوراً execute کریں
- Escalate ہو تو silent یا ambiguous کے لیے specific request کسی policy جب
- Tool results میں multiple matches کے additional identifiers وں تو

5.3 Multi-agent Systems میں Error Propagation Strategies کرنے

Key knowledge:

- Structured error context (failure type, query, partial results, alternatives) coordinator کو smarter recovery دیتا ہے
- Access failures (timeouts retry decision میں) اور valid empty results (no matches) میں فرق کریں
- Generic error statuses ("search unavailable") coordinator سے valuable context چھپا دیتا ہے
- Silent suppression یا failure پر پورا workflow abort کرنا anti-patterns میں

Key skills:

- Structured error context: failure type, کیا گیا attempt, partial results, possible alternatives واپس کریں
- Access failures کو valid empty results الگ کریں
- Transient failures subagents کے لیے صرف local recovery میں non-recoverable errors partial results کے ساتھ propagate کریں
- Synthesis میں coverage annotate کیا اچھی طرح supported اور gaps اور ان کے

5.4 Large Codebases Investigate کرتے وقت Context Efficiently Manage کرنا

Key knowledge:

- Long sessions میں context degradation: model unstable answers اور مخصوص classes دینے لگتا ہے
- Scratchpad files key findings کو context boundaries میں محفوظ رکھتے ہیں
- Subagents کو delegate سے verbose discovery output isolate کرنا
- Structured state persistence crash recovery ممکن بناتی ہے

Key skills:

- Specific questions subagents spawn میں رکھیں high-level coordination main agent کے لیے
- Key findings scratchpad files میں refer محفوظ رکھتے ہیں اور بعد میں
- Long investigations میں key findings summarize کرنا subagents spawn کے phase اگلا
- Long investigations میں context usage کم کرنا /compact استعمال کریں

5.5 Human Oversight اور Confidence Calibration کے ساتھ Workflows Design کرنا

Key knowledge:

- Aggregate metrics (97% overall accuracy) specific document types یا fields پر کمزوری چھپا سکتا ہے
- Stratified random sampling high-confidence extractions میں error rates measure کرنا

- Field-level confidence calibration labeled validation sets کے ذریعے
- Automation کے accuracy validate اور field segment کے حساب سے document type کے لیے

Key skills:

- New error patterns detect کے لیے stratified random sampling implement کریں
- Stable performance validate کے لیے field اور document type کے حساب سے accuracy analyze کریں
- Field-level confidence scores output کے review thresholds calibrate کریں اور labeled data کے ذریعے
- Low-confidence یا ambiguous-source extractions human review کی طرف route کریں

5.6 Multi-source Synthesis میں Provenance Preserve کرنا اور Uncertainty Handle کرنا

Key knowledge:

- Summarization کے دوران attribution کے "claim → source" mappings رکھو جاتی ہیں اگر
- Aggregation کے دوران structured mappings کے نیچے چاہئیں
- Conflicting statistics کو arbitrarily ایک value کے بجائے attribution منتخب کرنے کے ساتھ conflict annotate کر کے handle کریں
- Temporal differences کو contradiction کے لیے publication/collection dates سمجھنے سے بچنے کے لیے

Key skills:

- Subagents کے "claim → source" mappings (URL, document name, quotes) output کروائیں
- Reports کو stable findings اور disputed ones میں الگ کریں structure
- Conflicting values کو annotations کے ساتھ محفوظ رکھیں اور reconciliation کے لیے
- Correct temporal interpretation کے لیے publication dates شامل کریں
- Content type مطابق render کریں: financial data tables، news prose، technical findings، structured lists میں

Claude Certified Architect — Foundations Certification

مطالعہ کے گائیڈ (سرکاری امتحانی گائیڈ کی بنیاد پر)

تعارف

Claude Certified Architect — Foundations سرٹیفیکیشن اس بات کی تصدیق کرتی ہے کہ ایک ماہر Claude-based Claude Code، Claude Agent SDK، Claude API، اور Model Context Protocol (MCP) بنیادی ٹیکنالوجیز جن کے بنیادی علم کو جانچتا ہے — یعنی وہ trade-off حل بنانے کے وقت درست، یا امتحان کر سکتا ہے۔

اس کے ساتھ پروڈکشن ایپلیکیشنز تیار کی جاتی ہیں Claude کے

بنانا agentic systems امتحان کے سوالات حقیقی دنیا کے منظر ناموں پر مبنی ہوتے ہیں: کسٹمر سپورٹ کے لیے، multi-agent research pipelines، Claude Code کو CI/CD شامل کرنا، developer productivity، structured data بنانا، اور غیر ساختہ دستاویزات سے tools نکالنا

هدف امیدوار

کرتا آپ ship کے ساتھ پروڈکشن ایپلیکیشنز ڈیزائن اور Claude کے جو solution architect مثالی امیدوار ایک کے پاس کم از کم 6 ماہ کا عملی تجربہ ہونا چاہیے:

- Claude Agent SDK** — multi-agent orchestration، subagents کو delegate کرنا، tool integration، lifecycle hooks
- Claude Code** — CLAUDE.md، MCP servers، Agent Skills، planning mode
- Model Context Protocol (MCP)** — backend integration کے لیے tools اور resources
- Prompt engineering** — JSON schemas، few-shot examples، data extraction templates
- Context windows** — لمبی دستاویزات، multi-agent context passing
- CI/CD pipelines** — automated code review، test generation
- Escalation and reliability** — error handling، human-in-the-loop

امتحانی فارمیٹ

پیرامیٹر	قدر
سوال کی قسم	Multiple choice (میں سے 1 درست 4)
اسکورنگ	100–1000 scale، passing score 720
Guessing penalty	نہیں (سوال کا جواب دیں!)
Scenarios	8 4 (randomly selected) میں سے

امتحانی مواد: 5 ڈومینز

ڈومین	وزن
1. Agent architecture and orchestration	27%
2. Tool design and MCP integration	18%
3. Claude Code configuration and workflows	20%
4. Prompt engineering and structured output	20%
5. Context management and reliability	15%

امتحانی منظرنامہ

Scenario 1: Customer Support Agent

returns, billing disputes, اور account issues کو حل کرنے کے لیے Claude Agent SDK کے ساتھ agent بنانا ہے۔ آپ ایک agent MCP tools (get_customer, lookup_order, process_refund, escalate_to_human) کو 80% first-contact resolution کے ساتھ استعمال کرنا ہے۔ مناسب escalation کے ساتھ۔

Scenario 2: Claude Code کے ساتھ Code Generation

code generation, refactoring, debugging, documentation کے ساتھ Claude Code کو development کے لیے استعمال کرتے ہیں۔ آپ custom slash commands اور CLAUDE.md configuration کے ساتھ integrate کرنا ہے۔ planning mode کو سمجھنا ہے اور integrate کرنا ہے۔

Scenario 3: Multi-Agent Research System

tasks کو coordinator agent specialized subagents کے ساتھ سونپنا ہے: web research, document analysis, synthesis, اور report generation کے ساتھ۔ citations کے سسٹم کو report generation کے ساتھ۔

Scenario 4: Developer Productivity Tools

agent engineers کو unfamiliar codebases explore کرنے، boilerplate code اور routine tasks automate کرنے کے لیے MCP servers اور Built-in tools (Read, Write, Bash, Grep, Glob) کے ساتھ مدد دینا ہے۔

Scenario 5: Claude Code for Continuous Integration

pull اور automated code reviews, test generation, اور CI/CD pipeline کو Claude Code کے ساتھ۔ false positives کو کم کرنے کے لیے Prompts کو حاصل کرنے کے لیے۔

Scenario 6: Structured Data Extraction

JSON schemas کو validate کرنے کے لیے، output کو، سسٹم غیر ساختہ دستاویزات سے معلومات نکالنا ہے۔ درست طور پر سنبھالنے کے لیے edge cases پر برقرار رکھنا ہے۔

Scenario 7: Conversational AI Architecture Patterns

multi-turn conversational systems میں context window management، turns متعدد، memory strategies، execution محفوظ، tool design کے لیے، اور memory کا برقرار رکھنا ہے۔

Scenario 8: Agentic AI Tools (ہماری مدد کریں)

میں شامل ہیں۔ guide نہ دی لیکن ابھی تک اس candidates کی اطلاع امتحان دینے والے scenario اس میں share میں GitHub Issues کے سوالات دیکھیں، تو ان میں scenario اگر آپ اصل امتحان میں اس شامل کر سکیں۔ آپ کا تعاون اس شخص کی مدد کرے گا جو امتحان کی تیاری کر coverage تاکہ ہم مکمل کر سکیں۔

نظریاتی بنیادیں: حصہ 1

یہ حصہ و بنیادی نظریہ بیان کرتا ہے جس کی آپ کو امتحان میں کامیابی کے لیے ضرورت ہے۔ مواد کو topic کے مطابق — اس کے domains کے مطابق ترتیب دیا گیا ہے، نہ کہ امتحانی concepts اور technologies کی گہری سمجھ بھنی۔

کی بنیادیں Claude API — Model Interaction: باب 1

دستاویزات: [Messages API](#) | [Prompt Engineering](#)

1.1 API Request Structure

Claude API میں عموماً یہ Claude Messages API request پر کام کرتا ہے request-response model ایک شامل ہوتا ہے:

```
{
  "model": "claude-sonnet-4-6",
  "max_tokens": 1024,
  "system": "You are a helpful assistant.",
  "messages": [
    {"role": "user", "content": "Hi!"},
    {"role": "assistant", "content": "Hello!"},
    {"role": "user", "content": "How are you?"}
  ],
  "tools": [...],
  "tool_choice": {"type": "auto"}
}
```

م فیلڈز:

- `model` — model selection (`claude-opus-4-6`, `claude-sonnet-4-6`, `claude-haiku-4-5`)
- `max_tokens` — response میں زیادہ سے زیادہ tokens
- `system` — system prompt (model کی تعریف)
- `messages` — conversation history (full history بھیجنے ضروری ہے)
- `tools` — available tools کی definitions
- `tool_choice` — tool selection strategy

1.2 Message Roles

roles میں تین array `messages` ہوتے ہیں:

- `user` — user messages
- `assistant` — model responses (history میں شامل ہوتے ہیں)

- `tool` — tool call results (عملاً `tool_result` content block)

محفوظ state کے درمیان Model requests بھیجیں conversation history میں مکمل API request م بات: ر آزاد ہوتی call نہیں رکھتا؛ ر

1.3 stop_reason

Claude API response میں `stop_reason` کے جو بتاتا ہے کہ:

Value	Description	Action
"end_turn"	Model جواب مکمل کر لیا	کو نتیجہ دکھائیں user
"tool_use"	Model tool call چاہتا ہے	واپس دیں result چلائیں اور tool
"max_tokens"	Token limit ختم ہو گئی	response truncated؛ limit میں
"stop_sequence"	Stop sequence مل گئی	کریں handle کے مطابق application logic

کرتے loop control ہیں — یہی signals سب سے اہم "end_turn" اور "tool_use" میں Agentic systems ہیں

1.4 System Prompt

System prompt متعین کرتا ہے behavioral rules اور context کے جو instruction ایک خاص

- `messages` array الگ ہوتا؛ الگ `system` field کا حصہ نہیں ہوتا؛
- user messages پر فوقیت رکھتا ہے
- ہر پوری گفتگو میں لاگو رہتا ہے load ایک بار
- کی وضاحت کے لیے استعمال ہوتا ہے output format اور role, constraints,

کو غیر ارادی طور پر متاثر کر سکتی ہے مثال tool selection wording کی system prompt: امتحان کے لیے اہم ضرورت سے زیادہ `get_customer` کو model کی تصدیق کریں "جیسی ہدایت customer کے طور پر" ہمیشہ استعمال کرنے پر لے جا سکتی ہے

1.5 Context Window

Context window اس میں شامل process ایک وقت میں model کے جس (tokens) و کل متن

- System prompt
- message history مکمل
- Tool definitions
- Tool results

اہم مسائل:

1. **Lost-in-the-middle effect:** آغاز اور اختتام کی معلومات زیادہ مؤثر ہوتی ہیں، درمیان کی input لمبے details miss ہو سکتی ہیں حل: اہم معلومات شروع یا آخر میں رکھیں

2. **Tool results** واپس tool 40+ fields شامل کرتی ہیں اگر output میں tool call context کا جمع ہونا: ہر Tool results ضائع ہوتا ہے context کو مگر صرف 5 امیون تو
3. **Progressive summarization:** history compress کرنے پر numeric values, percentages, اور dates اکثر vague ہوتی ہیں

2 tool_use اور Tools: باب 2

Tool Use: دستاویزات

2.1 tool_use کیا ہے

براہ راست Model کو external functions call کرنے کے لیے Claude جس کے ذریعے mechanism tool_use واپس کرتا ہے result کو execute اس کے لیے code بنانا ہے: آپ کا structured tool call نہیں چلاتا — وہ code درست ہے

2.2 Tool Definition

tool JSON schema کے ذریعے define کیا جاتا ہے:

```
{
  "name": "get_customer",
  "description": "Finds a customer by email or ID. Returns the customer profile, including name, email, order history, and account status. Use this tool BEFORE lookup_order to verify the customer's identity. Accepts an email (format: user@domain.com) or a numeric customer_id.",
  "input_schema": {
    "type": "object",
    "properties": {
      "email": {"type": "string", "description": "Customer email"},
      "customer_id": {"type": "integer", "description": "Numeric customer ID"}
    },
    "required": []
  }
}
```

Tool description نکات:

1. Description کا سبب بنتی selection غلط Minimal descriptions mechanism کا بنیادی tool selection ہے
2. Description کا input format، کیا کرتا ہے، کیا واپس کرتا ہے tool میں شامل کریں
3. Overlapping descriptions سے بچیں
4. Built-in tools اور MCP tools overlap کو ہٹانے کے لیے MCP tool واضح بنائیں

2.3 tool_choice

Value	Behavior	کب استعمال کریں
<code>{ "type": "auto" }</code>	Model خود ط کرتا ہے	default عام
<code>{ "type": "any" }</code>	Model tool call کو لازماً کرے گی	structured output جب چاہے
<code>{ "type": "tool", "name": "extract_metadata" }</code>	Model tool call کو لازماً کرے گی	forced first step / order ہے

2.4 JSON Schemas for Structured Output

reliable لینے کا سب سے structured output Claude استعمال کرنا `tool_use` کے ساتھ JSON schemas کی semantic correctness نافذ کرتا ہے، مگر required structure کم کرتا ہے اور syntax errors طریقہ سے یقیناً ضمانت نہیں دیتا

اصول Design:

- رکھیں جو ہمیشہ ملنے چاہئیں required fields صرف وہ
- استعمال کریں nullable fields کے لیے missing data ممکن
- ضائع نہ ہو information شامل کریں تاکہ enum values جیسے "unclear" اور "other"

2.5 Syntax اور Semantic Errors

Error type	Example	Mitigation
Syntax	Invalid JSON, field type غلط	JSON schema کے ساتھ <code>tool_use</code>
Semantic	Totals match نہیں کرتے، value field غلط	Validation checks, feedback retry

بنانا Claude Agent SDK — Agentic Systems: باب 3

دستاویزات: [Agent SDK](#) | [Hooks](#) | [Subagents](#) | [Sessions](#)

3.1 Agentic Loop

چلانا sequence کی actions صرف جواب نہیں دیتا، بلکہ Claude میں Agentic loop

- Claude کو tools کے ساتھ request بھیجیں
- Response لیں
- چیک کریں `stop_reason`:
 - "tool_use" → tool چلائیں، result history پھر دوبارہ request میں ڈالیں

- "end_turn" → task کو دین user مکمل، نتیجہ task

مکمل ہونے تک دہرائیں۔ 4.

Text parsing یا arbitrary iteration دیکھیں کہ `stop_reason == "end_turn"` کہ لیں `signal: completion` درست قابل اعتماد نہیں ہیں۔ limit

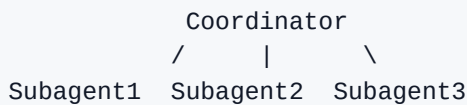
3.2 AgentDefinition

```
agent = AgentDefinition(
    name="customer_support",
    description="Handles customer requests for returns and order issues",
    system_prompt="You are a customer support agent...",
    allowed_tools=["get_customer", "lookup_order", "process_refund",
"escalate_to_human"],
)
```

اصول اپنائیں کہ Least privilege کہ allowed_tools اور، system_prompt، description، name: کم چیزیں

3.3 Coordinator اور Subagents

استعمال کرتی ہیں hub-and-spoke topology اکثر Multi-agent systems:



Coordinator task کو توڑتا کہ، subagents نتائج جوڑتا کہ، کام سونپتا کہ، منتخب کرتا کہ، errors handle کرتا کہ، جواب پہنچانا کہ final تک user کہ، اور

نہیں کرتے کہ inherit خود بخود parent history الگ ہوتا کہ؛ وہ کہ context کا subagents: کم اصول

3.4 Task Tool

Subagents Task tool کے ذریعے spawn کہیں کہ Coordinator کہ allowed_tools میں "Task" ہونا کہ چاہیے کہ

context passing: صحیح

- Task: "Analyze the document" کافی نہیں
- Task: "Analyze this document. Full text: ... Output schema: ..." درست کہ

Coordinator ایک response میں multiple Task s کہ، parallel subagents بھیج سکتا کہ، لہذا کہ

3.5 Hooks

Hooks lifecycle کے مخصوص points پر work کہیں کہ کرتے کہ

PostToolUse tool result کو model پر لے کر normalize تک پہنچانے کے لیے PostToolUse tool result کو model پر لے کر normalize تک پہنچانے کے لیے:

```
@hook("PostToolUse")
def normalize_dates(tool_result):
    return normalized_result
```

PreToolUse policy violation block کر سکتا ہے:

```
@hook("PreToolUse")
def enforce_refund_limit(tool_call):
    if tool_call.name == "process_refund" and tool_call.args.amount > 500:
        return redirect_to_escalation(tool_call)
```

Critical business rules کو یقینی بنانے کے لیے hooks deterministic؛ prompt instructions probabilistic؛ یاد رکھیں: hooks کو یقینی بنانے کے لیے rules کو یقینی بنانے کے لیے:

باب 4: Model Context Protocol (MCP)

[MCP](#) | [Tools](#) | [Resources](#) | [Servers](#) : دستاویزات

4.1 MCP کیا ہے

MCP resource سے جوڑتا ہے اس کے تین بنیادی Claude کو external systems کو جو open protocol ہے:

1. **Tools** — actions کے لیے functions
2. **Resources** — context data (documentation, schemas, catalogs)
3. **Prompts** — predefined task templates

4.2 MCP Servers

MCP server ایک process جو tools/resources expose کرتا ہے Connect کے ذریعے tools discover ہوتا ہے اور descriptions کے مطابق استعمال ہوتا ہے

4.3 Configuration

Project config (`.mcp.json`) لے کر:

```
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {"GITHUB_TOKEN": "${GITHUB_TOKEN}"}
    }
  }
}
```

User config سہ آتی ہے secrets environment variables میں ہوتی ہے؛ Project config version control (~/claude.json) personal/experimental servers کے لیے ہوتی ہے

4.4 isError

واپس کریں structured error کے بجائے Generic errors استعمال کریں `isError: true` میں MCP tool errors

```
{
  "isError": true,
  "content": {
    "errorCategory": "transient",
    "isRetryable": true,
    "message": "The service is temporarily unavailable."
  }
}
```

4.5 Resources

Resources کے پڑھ سکتا ہے action بغیر agent میں جو data و Resources: schemas, docs, content catalogs, summaries، exploratory tool calls کے لیے کم کرتے ہیں

5 اور Claude Code — Configuration Workflows

دستاویزات: [Claude Code](#) | [Memory / CLAUDE.md](#) | [Skills](#) | [MCP](#) | [Hooks](#) | [Sub-agents](#) | [GitHub Actions](#) | [Headless](#)

5.1 CLAUDE.md Hierarchy

تین سطحوں پر ہو سکتا ہے CLAUDE.md

- **User-level:** `~/.claude/CLAUDE.md` — پر لاگو user صرف اسی
- **Project-level:** `.claude/CLAUDE.md` یا `root` `CLAUDE.md` — لی تیم پوری
- **Directory-level:** subdirectories میں `CLAUDE.md` — folder conventions مخصوص

5.2 @path Syntax

CLAUDE.md میں @path سے external files refer تاکہ جا سکتا ہے میں تاکہ:

Coding standards `@./standards/coding-style.md` میں ہیں
Test rules `@./standards/testing-requirements.md` میں ہیں

5.3 .claude/rules/

`.claude/rules/` rules کو topic سے لحاظ سے organize کرنا ہے اور `paths` ذریعہ conditional loading دیتا ہے:

```
---
paths: ["src/api/**/*"]
---
```

استعمال کریں explicit error handling اور async/await کہ لہ API files

5.4 Commands اور Skills

`.claude/commands/` project commands جبکہ `~/.claude/commands/` personal commands کے لیے ہے، جیسے `.claude/skills/` میں frontmatter کے ساتھ skills کے ملتی ہیں

5.5 Skills اور context: fork

`context: fork` skill کو isolated context میں چلاتا ہے `allowed-tools` اور tool access سے محدود کریں، `argument-hint` مانگیں `required input` سے

5.6 Planning Mode

Planning mode کے لیے بہترین unfamiliar codebases اور multiple approaches، تبدیلیوں architectural بڑی Direct execution کے لیے بہتر changes چھوٹے اور واضح

5.7 /compact اور /memory

`/compact` context compress کرنا ہے، اور `/memory` notes اور preferences میں مدد دیتا ہے

5.8 Claude Code CLI for CI/CD

ک: لے استعمال ہوتا ہے non-interactive mode / -p --print

```
claude -p "Analyze this pull request for security issues"
```

Structured output لے لے --output-format json اور --json-schema استعمال کریں

باب 6: Prompt Engineering — Advanced Techniques

دستاویزات: [Prompt Engineering](#) | [Anthropic Cookbook](#)

6.1 Few-shot Prompting

Few-shot prompting 4–2 examples میں کر expected behavior یں دکھایا جاتا ہے۔ Vague instructions سے کم کرتا example ambiguity زیادہ مؤثر ہے کیونکہ

6.2 Explicit Criteria

Vague instructions بجائے clear criteria کیا flag دیں: ignore کرنا، یا severity کرنا، اور decide کیسے کرنا۔

6.3 Prompt Chaining

Complex task کو focused steps میں توڑیں: local analysis، پھر integration pass، یا attention dilution۔

6.4 Interview Pattern

Claude domain سامنے لاتا، خاص طور پر جب design ambiguity پوچھوana clarifying questions سے unfamiliar ہو۔

6.5 Validation and Retry

Extraction fail ہو تو original document، incorrect output، اور specific validation error ساتھ retry کرنا۔

6.6 Self-correction

کریں discrepancy handle تو conflict دونوں نکال؛ Model stated value اور calculated value

7: Message Batches API

دستاویزات: [Message Batches](#)

7.1 Overview

Message Batches API asynchronous processing 50 :% savings، up to 24 hours window، اور multi-turn tool calling supported ، custom_id requests اور responses link میں

7.2 کب استعمال کریں

Pre-merge checks اور interactive tasks ، synchronous API، overnight reports اور bulk jobs کب استعمال کریں Batch API

7.3 custom_id

custom_id result کو original document ، failures identify اور صرف failed items دوبارہ submit میں مدد دیتا

8: Task Decomposition Strategies

8.1 Fixed Pipelines

Predictable tasks ، fixed pipeline جسے Document → Extraction → Validation → Final output

8.2 Dynamic Decomposition

Open-ended tasks میں subtasks intermediate results کی بنیاد پر بنتے ہیں

8.3 Multi-pass Code Review

Large PRs ، per-file analysis بعد cross-file integration pass ، attention dilution کریں تاکہ

9 Escalation اور Human-in-the-Loop: باب 9

9.1 Escalate کب کریں

Explicit user request, policy gaps, progress نہ ہونا, threshold سے اوپر financial action, یا ambiguous matching میں Sentiment analysis اور self-rated confidence unreliable ہیں۔ Escalate کی صورت میں

9.2 Escalation Patterns

human کو hand off نہ کرنا یا explicit insistence یا policy gap نہ کرنے کی کوشش کریں، پھر اگر resolve نہ ہو، مسئلہ حل کریں۔

9.3 Structured Handoff

Escalation پر human شامل کریں تاکہ recommended action اور customer ID, issue summary, actions taken, اور context کو مکمل operator

9.4 Confidence Calibration

Field-level confidence scores, calibration on labeled data, اور stratified sampling استعمال کریں۔

10 Error Handling میں Multi-agent Systems: باب 10

10.1 Error Categories

Transient errors retryable ہیں، validation errors input fix ہیں، مانگتی business errors policy-driven ہیں، اور permission errors generally escalate ہیں، اور

10.2 Anti-patterns

Generic “search unavailable” response, silent suppression, workflow abort یا پورا کرنا نقصان دہ۔ Structured errors واپس کریں۔

10.3 Structured Subagent Error

Structured error میں failure type, attempted query, partial results, alternative approaches, اور coverage impact شامل کریں۔

10.4 Final Synthesis میں Coverage

پارٹیاں، اور کچھ partially covered ہیں، کون سی sections fully covered میں واضح کریں کہ کون سی Report باقی ہیں۔ gaps

11 Context میں Production Systems: باب 11 Management

11.1 Facts Separate Block میں

Transactional facts کو persistent case facts block تاکہ summarization میں رکھیں تاکہ critical data کے باوجود زائد و اضافہ؛

11.2 Tool Results Trimming

PostToolUse hook رکھیں تاکہ relevant fields سے صرف

11.3 Position-aware Input

Important findings اور action items شروع میں اور lost-in-the-middle effect آخر میں رکھیں تاکہ

11.4 Scratchpad Files

Verbose findings scratchpad تاکہ long investigations میں لکھیں تاکہ context محفوظ رہے

11.5 Subagents کو Delegate کرنا

Exploration subagents میں summary میں صرف main context میں دیں اور

11.6 Structured State Persistence

ممکن ہے کہ crash recovery تاکہ export میں JSON file یا state manifest اپنی agent سے

12 Provenance رکھنا: باب 12 کو محفوظ رکھنا

12.1 Attribution Loss

Claim رکھیں تاکہ source URL، publication date، اور confidence کے ساتھ

12.2 Conflicting Data

کریں conflict flag ساتھ محفوظ رکھیں اور attribution دیں تو دونوں کو values مختلف sources اگر

12.3 Dates

دیکھیں publication dates سمجھنا سہل ہے contradiction کو Temporal differences

12.4 Content Type مطابق Render

Financial data tables میں، news prose میں، technical findings structured lists اور time series میں دیکھائیں chronological order

13 Claude Code کے Built-in Tools: باب 13

13.1 Tool Selection Reference

Task	Tool	Example
تلاش کرنا file سہ pattern	Glob	<code>**/*.test.tsx</code>
file contents میں search	Grep	function name, error message
پڑھنا file	Read	analysis کے لیے
بنانا file نیا	Write	scratch سہ
precise edit	Edit	unique text match
shell command	Bash	git, npm, tests

13.2 Incremental Investigation

دیکھیں consumer files پھر، usages پھر، entry points نہ پڑھیں؛ لہذا files ایک ساتھ سب

13.3 Read + Write Fallback

کریں Write کریں، پھر programmatically کریں، تبدیلی Read کو تو Edit fail اگر

امتحانی ڈومین نوٹس: II حصہ

1 ڈومین: Agent Architecture اور Orchestration (27%)

Agentic loops, coordinator–subagent architecture, Task ذریعہ spawning, hooks, escalation, اور multi-step workflows پر فوکس کریں

2 ڈومین: Tool Design اور MCP Integration (18%)

Tool descriptions, tool_choice, JSON schema output, structured errors, MCP servers, resources, اور built-in tools کے فرق کو سمجھیں

3 ڈومین: Claude Code Configuration اور Workflows (20%)

CLAUDE.md hierarchy, .claude/rules/, commands, skills, planning mode, context: fork, اور CI/CD integration

4 ڈومین: Prompt Engineering اور Structured Output (20%)

Few-shot prompting, explicit criteria, prompt chaining, validation/retry loops, JSON schema enforcement, اور batch-friendly prompting پر توجہ دیں

5 ڈومین: Context Management اور Reliability (15%)

Summarization risks, lost-in-the-middle, trimming tool output, confidence calibration, escalation, اور provenance preservation یاد رکھیں

Scenario 7: Conversational AI Architecture Patterns

سوال 61

کرنے کے لیے preview سے پہلے اثرات کا execution میں tool remove_team_member صورتحال: آپ کے براہ راست agent سے معلوم ہوتا ہے کہ Production monitoring کے parameter dry_run: boolean سے dry_run=false ساتھ call کے ساتھ bypass کو preview step کو کر کے call کے ساتھ deletion ایک preview سے پہلے user کو explicitly confirm سے پہلے approach قابل اعتماد

کونسی؟ approach سب سے قابل اعتماد

- A) Server-side validation شامل کریں جو dry_run=false سے dry_run=true call سے seconds میں parameters کے ساتھ
- B) Tool کو configuration layer اور orchestration کی ضرورت کے طور پر confirmation کو annotated tools کے approval کی user کرنے سے پہلے calls forward کے

- C) Tool description agent شامل کریں جن میں few-shot examples اور instructions میں تفصیلی `dry_run=true` کو `call` سے `replace` کریں `preview_remove_member` impact details اور single-use confirmation token کو `preview` سے `bind`، درکار `token` کو `execute_remove_member` واپس کرتا ہے؛ `execute_remove_member` [درست] کرتا ہے

D) Token-binding approach سے `architecturally` `preview` ناممکن ہو جاتا ہے `execute tool` کو `literally` `execute` — `execute tool` جو صرف `token` `generate` `preview tool` `orchestration infrastructure` یا (A)، `timing heuristics` (C)، `LLM instructions` جو `approach` `code level` پر `constraint enforce` `code level` پر انحصار کے بجائے (B) کرتی ہے

سوال 62

`search_catalog` tool 12% fail وقت `Production monitoring` کا آپ کا `search_catalog` سے معلوم ہوتا ہے کہ آپ کا `search_catalog` `query syntax errors` % پر کامیاب ہوتا ہے، اور `retry 4` میں جو `network timeouts` %: 8 `error types` `return` یکساں طریقہ سے `error types` کامیاب نہیں ہوتا ہے فی الحال دونوں `retries` میں ہوتی ہیں

کریں گے؟ `modify` کیسے `error handling` کی tool آپ

- A) System prompt میں few-shot examples شامل کریں جو `network errors` میں فرق کیسے کریں `syntax errors`
- B) `uniformly exponential backoff retry logic` apply کریں `errors` پر تمام
- C) Tool `network timeouts` کے اندر `automatic retry with backoff` implement کریں؛ `syntax errors` کے ساتھ فوری واپس کریں [درست] `parameter validation details`
- D) `retryable` boolean flag اور `error type details` کو `errors` تمام

C) Tool level پر `transient errors` کے لیے `retries` handle کرنا صحیح `abstraction boundary` — tool `error type` پر `agent` کر سکتا ہے بغیر `deterministic retry logic` کے بارے میں یقینی علم اور `error type` `Uniform backoff` (B) `syntax errors` `prompt instructions` (A) follow `interpret` (D) `flag` انحصار کے `error type` `errors` پر وقت ضائع کرتا ہے جو کبھی کامیاب نہیں ہوتے ہوں گے

سوال 63

بہت کم "اور `risk tolerance` نہ `user`، میں `turns` کی گفتگو کے کئی `Investment strategy` `returns maximize` پھر "میں اپنے `invest` کرنا چاہتا ہوں؟" اب وہ پوچھتا ہے: "مجھے کس چیز میں `returns maximize` پھر "میں اپنے

سے `priority` کی اصل `recommendation user` سب سے بہتر طریقہ سے یقینی بنانا ہے کہ `approach` کونسا `مآئینگ` ہے؟

- A) پوچھیں کہ کونسی چیز زیادہ `مآئینگ` [درست] `user` تضاد کو سامنے لائیں اور
- B) `recommendations` کے لیے الگ الگ `scenarios` دونوں
- C) `preference` سب سے `حال` میں بیان کی گئی
- D) `balanced portfolio` تضاد کو حل کیے بغیر

مانگنا کی واحد clarification برا راست متضاد ہوں، تو تضاد سامنے لانا اور user preferences کی کیوں: جب A low risk کرنا اور Returns maximize کے اصل ارادہ سے ہم آہنگ ہو۔ user recommendation طریقہ کے بنیادی طور پر ناقابل مطابقت ادا ف میں جن کے لیے انسانی فیصلہ درکار کے tolerance

سوال 64

jazz کے "مجھے User کرتے ہیں playlist preferences refine میں conversation turns کئی Users: صورتحال "پسند ہیں؟ genres پوچھتا ہے "آپ کو کون سے Claude، بعد messages پسند ہے" کے دو

سب سے زیادہ ممکنہ وجہ کیا ہے؟

- A) Claude conversation memory کے لیے vector database connection درکار ہے
- B) Model کی context window exceed ہو گئی ہے
- C) Claude API کو session_id parameter درکار ہے
- D) شامل نہیں کر رہی ہے [درست] messages میں بچھلے messages array application آپ کی

request کے stateless API call ہیں — server-side memory کے پاس Claude: کیوں D کا کوئی علم turns کے پاس بچھلے Claude، شامل کیے بغیر conversation history میں مکمل messages array کا حصہ نہیں ہیں؛ دو architecture کے Claude (C) session_id اور (A) Vector databases ہیں ہوتا ناممکن (B) context window overflow کے لیے exchange کے messages

سوال 65

History تک پہنچ گئی ہے conversation 78,000 tokens، بعد cooking session صورتحال: 40 منٹ کے شامل ہیں آپ کو اسم معلومات discussion اور عمومی، allergies, recipe scaling, clarified cooking terms، کم کرنے ہیں tokens محفوظ رکھتے ہوئے

کے درمیان بہترین توازن رکھتا ہے؟ token reduction اور approach preservation کونسا

- A) خلاصہ کریں conversation history پوری
- B) رکھیں tokens صرف سب سے حالیہ 20,000
- C) خلاصہ کریں، اور discussion نکالیں، عمومی (allergies, quantities, preferences) structured data اسم رکھیں [درست] verbatim کو exchanges حالیہ
- D) کریں retrieve کے ذریعہ متعلقہ حصہ semantic search کریں اور conversation externally store مکمل

Allergies معلومات محفوظ رکھتی ہے valuable پر سب سے زیادہ cost سب سے کم Hybrid approach: کیوں C (summarization) ہوتا ہے میں compact structured block ایک critical facts جیسے recipe quantities اور exchanges ہوتی ہے، اور حالیہ discussion summarize عمومی، (سہ بچاتے ہوئے precision loss کے دوران conversational coherence کے لیے verbatim میں Options A اور B dietary information اسم کے لیے ضرورت سے زیادہ پیچیدہ ہے cooking session ایک D ہیں؛

سوال 66

ka preferences اور topics پر assistant کے conversations میں reports کرتے ہیں کہ طویل Users: صورتحال رکھتی ہے message pairs صرف آخری 25 implementation حساب کھو دیتا ہے آپ کی موجود

کیا ہے؟ solution سب سے مؤثر

- A) Hybrid approach: messages پرانے حالیہ کر کے خلاصہ کر کے دے کر messages verbatim [درست]
- B) vector similarity search پر conversation history مکمل
- C) Window 50 message pairs تک بڑھائیں
- D) آگے جوڑیں running summary کا خلاصہ کریں اور dropped messages میں turn کر

A: Hybrid approach کیوں؟ محفوظ رکھتی ہے exact recent context کی مسئلہ کہ دونوں پر لوؤں کو حل کرتی ہے Hybrid approach کیوں؟ A برقرار compressed representation کی preferences جبکہ پر لے کر (کے لیے) م (conversational coherence) miss کو context م (B) Vector search صرف اسی مسئلہ کو ملتوی کرتا ہے (C) بڑھانا Window رکھتی ہے م (D) semantically سے current query کر سکتی ہے جو

سوال 67

بڑھ جاتی latency سے زیادہ ہوتے ہیں تو 50 turns conversations رپورٹ کرتے ہیں کہ جب Users: صورتحال بھی costs اور

بنیادی وجہ کیا ہے؟

- A) شامل ہوتی ہے conversation history کے ساتھ پوری API request کر
- B) Model gradually responses generate کرتا ہے
- C) History سے ساتھ database operations میں بڑھنے کے ساتھ
- D) Model بناتا ہے internal user profile کی ضرورت والا processing زیادہ

A: Claude کی API پر مکمل طور پر stateless — request کو messages array میں پوری request بڑھتی ہے، conversation history شامل کرنی ہوتی ہے جسے جسے conversation history کے درمیان کوئی Model calls دونوں بڑھاتا ہے cost اور directly processing latency ل جاتی ہے، جو internal state رکھتا ہے (D غلط ہے)

سوال 68

تک بڑھ گئی ہے جب conversation history 85,000 tokens، بعد weekly sessions: صورتحال: تین مہینوں کے discussions پچھلی assistant کے بارے میں کیا نتیجہ اخذ کیا تھا؟، تو theme کے isolation پوچھتا ہے "م نہ user کر کے کے بجائے عام جوابات دیتا ہے reference

کیا ہے؟ approach سب سے مؤثر

- A) Rolling window truncation
- B) Key conclusions capture کر کے دے کر progressive summarization
- C) Relevant exchanges retrieve کر کے لے کر [درست] semantic embeddings
- D) Discussion conclusions mark کر کے لے کر structured XML tags

C scale discussion کو جو تین مہینوں کی approach واحد semantic search پر conversation history پر **کیوں:**

Rolling window کر سکتی surface specific relevant exchanges demand پر کر سکتی جبکہ truncation (A) history کو Progressive summarization (B) discussions کا بیشتر حصہ ضائع کر دے گی۔

پوچھنے والے users کھو دیتی ہیں جن کے بارے میں specific conclusions، کرتی ہے compress میں abstractions

سوال 69

کرتا ہے، system prompt guidelines میں 10-15 Claude، دوران QA testing: صورتحال
 اندر 1000 token limits ابھی Conversation آجائی 1000 deviation میں responses لیکن بعد کے

کیا ؟ solution بہترین

- A) Behavioral guidelines میں منتقل کریں □ user message کو پلا □
- B) 20 turns conversation شروع کریں □ بعد نئی □
- C) Conversation breakpoints پر guidelines reinforce □ user-role messages insert □ [درست]
- D) Non-compliant responses کو regenerate □ post-response validation □ کریں □

history کا براہ راست مقابلہ کرتا ہے Behavioral reminders کا periodic injection instruction drift کو regular intervals پر constraints re-establish کرتا ہے Guidelines کو user کو پر Accumulate message کو Post-response validation (D) corrective میں منتقل کرنا authority ان کا (A) preventive اور significant latency شامل کرتی ہے

سوال 70

□□ teaching methodology اور adaptation rules □□ جو AI tutor 2,800 token system prompt میں **صورتحال:** آپ کو define 12 □□□ □□□ assistant proficiency levels □□□ کر دیتا □□، بعد 12 turns کرتا □□□

کیا fix سب سے مؤثر

- A) 5–4 turns میں inject کریں
- B) Verbose rules کو proficiency-level adaptation demonstrate کرنے والے few-shot examples سے replace کریں [درست]
- C) Critical rules کو system prompt میں رکھیں
- D) Responses evaluate کریں اور اگر difficulty level match نہ ہو تو regenerate کریں

B کیوں: Declarative rules 2,800 والا token system prompt drift کیونکہ abstract rules vulnerable لی concrete few-shot examples کو Verbose rules کا تقاضا کرتے ہیں reasoning model میں turn ر clear کرنا لی match کو model ، کریں demonstrate proficiency-level adaptation correct کرنا جو replace behavioral patterns reliably follow میں ، turns دیتا — ، کئی behavioral patterns

سوال 71

clarifying کرنا، اور reasoning explain برقرار رکھنا، اپنی enthusiastic tone کو assistant **صورتحال**: آپ کے □
□ ہونی چاہئیں؟ define اس behavioral guidelines یوجہ □ □ ہیں □ □ questions

سوال 74

Assistant 4+ clarifying requests میں "جیس" venue book اکثر "پارٹی" لیں Users: صورتحال
abandonment % پوچھتا ، جس سے 35 questions

کرتا ؟ improve کو بہترین طریقہ سے approach trade-off کونسا

- A) Hidden defaults سے بڑھیں
- B) میں پوچھیں compound message ایک clarifying questions تمام
- C) Assumptions explicitly بیان کریں اور [درست] بڑھیں
- D) Structured intake form استعمال کریں

دیتا جبکہ غلط response کو فوری، مفید user بیان کرنا اور آگے بڑھنا Assumptions explicitly: کیوں C
کو نہیں بتاتا کیا (A) user Hidden defaults کرنے کی صلاحیت برقرار رکھتا correct کو assumptions
Structured کا تقاضا کرتی upfront effort سے user پھر بھی (B) Compound question list کیا گیا assume
form (D) کی بجائے زیادہ شامل کرتا friction کم

سوال 75

turns rules follow استعمال کرتا ابتدائی assistant contractor-persona system prompt صورتحال: آپ کا
Conversational length 2,500 tokens صرف دیتا assistant generic advice تک 7 turn کرتے ہیں، لیکن

سب سے زیادہ ممکنہ وجہ کیا ہے؟

- A) System prompts initial behavior establish کرتے ہیں صرف
- B) Turns accumulate ساتھ ساتھ model کو
- C) Accumulated assistant responses system prompt اثر کو dilute [درست] میں
- D) System prompt صرف ایک بار بھیجا جاتا ہے

System prompt جمع ہوتا ہے assistant responses میں conversation history کیوں: جیس جیس C
behavioral constraints کو reflect والے text کا proportion ہوتا ہے assistant-generated content بڑھتا ہے
بجائے اپنے پلے system prompt instructions پر بھی token lengths چھوٹے Model کم ہوتا جاتا ہے relative
outputs کرتا compound کو drift، کرتا follow کو زیادہ patterns کے

سوال 76

کئی Assistant "میں مدد کر سکتے ہیں؟" report کرتے ہیں جیس "کیا آپ requests میں Users: صورتحال
abandonment % جس سے 40، (کیا؟ deadline؟ کیا مدد؟ report؟ کون سی) کرتا respond سوالات پوچھ کر
ہوتا

کیا solution بہترین

- A) Reasonable assumptions انہیں اور explicitly کریں،
- B) Respond سے پلے smaller model ساتھ سے
- C) Assumptions کریں predefined interpretations بیان کیے بغیر
- D) Assistant تک محدود کریں clarifying question میں ایک turn کو

رکھتے ہیں اور in control کو user کو informed کے ساتھ آگے بڑھنا Reasonable stated assumptions: کیوں A کرتی ہے confuse کو users (C) Silent predefined interpretations کو مکمل طور پر ختم کرتا ہے back-and-forth کی back-and-forth turns بھی (D) One-question limit کے ساتھ match ان کے ارادے سے response میں جب latency اور infrastructure مسئلہ حل کے بغیر core UX (B) Smaller classification model ضرورت ہے شامل کرتا ہے complexity

عملی مشقیں

Exercise 1: Multi-tool Agent with Escalation Logic

Goal: tool integration, structured errors, اور escalation کے ساتھ agent loop بنائیں

Exercise 2: Configuring Claude Code for Team Development

Goal: CLAUDE.md, custom commands, path-specific rules, اور MCP servers configure کریں

Exercise 3: Structured Data Extraction Pipeline

Goal: JSON schema, validation/retry loops, اور Message Batches API کے ساتھ extraction pipeline بنائیں

Exercise 4: Designing and Debugging a Multi-agent Research Pipeline

Goal: subagent orchestration, explicit context passing, error propagation, اور source tracking implement کریں

Appendix: Technologies and Concepts

Technology	Key aspects
Claude Agent SDK	agent loops, <code>stop_reason</code> , hooks, <code>Task</code> , <code>allowedTools</code>
MCP	servers, tools, resources, <code>isError</code> , <code>.mcp.json</code>
Claude Code	CLAUDE.md, <code>.claude/rules/</code> , <code>.claude/commands/</code> , <code>.claude/skills/</code> , planning mode
Claude Code CLI	<code>-p</code> , <code>--output-format json</code> , <code>--json-schema</code>
Claude API	<code>tool_use</code> , <code>tool_choice</code> , <code>stop_reason</code> , <code>max_tokens</code>

Message Batches API	50% savings, 24-hour window, <code>custom_id</code>
JSON Schema	required/optional, nullable, enum values
Few-shot prompting	ambiguous scenarios ❌ لے ❌ examples
Prompt chaining	sequential decomposition
Context window	token budget, lost-in-the-middle
Confidence calibration	field-level scoring, stratified sampling

Out-of-Scope Topics

❌: موضوعات امتحان میں شامل نہیں ہیں

- Claude models کی fine-tuning
- Authentication, billing, یا account management
- implementation کی تفصیلی programming languages مخصوص
- hosting/infrastructure کی MCP servers
- Claude کی internal architecture یا model weights
- Constitutional AI, RLHF, یا safety training
- Embeddings یا vector databases کی implementation details
- Computer use / browser automation
- Vision / image analysis
- Streaming API یا SSE
- Rate limiting اور detailed API cost calculations
- OAuth اور API key rotation details
- Cloud-provider-specific configs
- Performance benchmarks
- Prompt caching implementation details
- Token counting algorithms

Preparation Recommendations

1. بنائیں agent کو ساتھ ایک Claude Agent SDK
2. کریں configure پر real project کو Claude Code
3. کریں test اور MCP tools design
4. بنائیں Data extraction pipeline
5. کریں Prompt engineering practice
6. سیکھیں Context management patterns

7. Escalation اور human-in-the-loop workflows سمجھیں

8. Practice exam حل کریں